IMPERIAL

Imperial College London

MSc in Applied Mathematics

MSc Project

Optimal Control of the Poisson and Burgers Equations, with a Study of Mesh-Dependence in the Optimisation

---

Author:

Bert Jim Chang

bert-jim.chang25@imperial.ac.uk

Supervisor:

Professor David A. Ham

Department of Mathematics

Faculty of Natural Sciences

July 28, 2026

Abstract

This dissertation studies mesh dependence in PDE-constrained optimisation. The number of iterations that a gradient-based optimiser needs can either stay flat or grow without bound as the computational mesh is refined, depending only on the inner product the optimiser uses to turn a derivative into a gradient. We first derive and verify the adjoint-based optimisation machinery for two model problems, the linear Poisson equation and the nonlinear, time-dependent viscous Burgers equation, using manufactured solutions and Taylor tests. We then reproduce the central result of Schwedes et al. [13]: respecting the function-space L^2 inner product through its Riesz map makes a quasi-Newton optimiser essentially mesh-independent, while the Euclidean inner product used by default in most optimisation packages does not. The main contribution of this work is to demonstrate that the same mesh dependence appears again one level deeper, in the inner Krylov solve that a Newton optimiser uses to compute each step, and to remove it there with the same Riesz map applied as a preconditioner. That the inner product acts as a preconditioner is anticipated by classical operator-preconditioning theory and by ongoing PETSc/TAO development, but we appear to be the first to expose and cure the effect experimentally in an adjoint optimisation pipeline. The metric must be injected at both the outer and the inner level. We give a spectral explanation of why the Riesz map is the right preconditioner, show that it must match the norm in which the control is regularised, and describe a productionised version of the preconditioner that has been prepared for contribution to the pyadjoint library.

Acknowledgements

David, Josh, PETSc discord,

Declaration of Originality

The work contained in this thesis is my own work unless otherwise stated.

Contents

1 Introduction

Many problems in science and engineering involve determining unknown inputs to a physical system based on partial or indirect observations of its output. These sorts of problems are commonly formulated as PDE-constrained optimisation problems, in which the behaviour of the system is governed by a partial differential equation, while the unknown input is found by minimising a suitable objective functional.

These governing equations are partial differential equations, and solving one on a computer almost always means the finite element method. The idea is to replace the continuous domain by a *mesh*, a partition of it into many small cells, here triangles, and to approximate the unknown function by a simple polynomial on each cell. This turns the differential equation into a finite system of linear equations, with roughly one unknown per mesh vertex, which a computer can solve directly, and the approximation improves as the mesh is made finer. Importantly for what follows, practical meshes are deliberately *non-uniform*: they are refined where the solution changes rapidly and left coarse elsewhere. That a mesh can be graded in this way, with small cells in some regions and large cells in others, is exactly what makes mesh-dependence a phenomenon worth studying.

Solving an optimisation problem on top of such a finite element model needs several ingredients, which we build in order. The first is a *forward solve*: given a trial control, it solves the PDE and returns the resulting state. The second is the *gradient* of the objective with respect to the control, which the optimiser needs in order to know which way to step; computing it directly, one direction at a time, would be far too expensive, so we use the *adjoint* method, which recovers the entire gradient from a single additional solve. The third is the *optimiser* itself, which uses the gradient to iterate towards the best control. Each of these has to be built and, just as importantly, verified before it can be trusted, which is why the first half of this dissertation derives and tests the forward solve, the adjoint and the gradient on two model problems before any mesh-dependence experiment is attempted. The final ingredient, and the one the whole story turns on, is the *inner product* the optimiser uses to turn the gradient into a search direction.

Such problems are solved iteratively, and one would expect that refining the computational mesh simply sharpens the answer without changing the character of the optimisation itself. This expectation can fail dramatically. On the same problem, solved by the same quasi-Newton method to the same tolerance, the number of iterations needed can either stay essentially constant as the mesh is refined or grow without bound: in the experiments of this dissertation, a flat count of around eight against a count that doubles at every refinement, reaching several hundred. The difference is not the problem, the algorithm, or the mesh, but a single choice buried inside the optimiser: the inner product it uses to turn a derivative into a gradient. Most optimisation packages are hard-wired to the Euclidean inner product of the coefficient vector, which is the wrong choice on a non-uniform mesh. The right choice is the inner product of the function space in which the control actually lives, applied through its Riesz map. This dissertation is really the story of that one choice.

We approach it in three stages. First, we build and verify the adjoint-based optimisation machinery on two model problems, the linear Poisson equation and the nonlinear, time-dependent Burgers equation, so that the later experiments rest on a pipeline whose forward solves and gradients are known to be correct. Second, we reproduce the central mesh-dependence result of Schwedes et al. [13]: respecting the function-space inner product makes a quasi-Newton optimiser essentially mesh-independent, while the Euclidean inner product does not. Third, we expose the same mesh-dependence one level deeper, in the inner Krylov solve that a Newton optimiser uses to compute each step, and remove it by the same idea applied there, as a preconditioner. That the inner product plays the role of a preconditioner is anticipated by classical operator preconditioning [6, 8] and has been discussed among the PETSc/TAO developers [7]; the

contribution here is to expose and cure the effect experimentally in an adjoint optimisation pipeline, to explain it spectrally, and to prepare a productionised preconditioner for the pyadjoint library. The metric has to be put in at both levels.

The dissertation is organised as follows. Section 2 collects the functional-analytic background (Sobolev spaces, dual spaces, and the Riesz representation theorem) on which the whole argument rests. Sections 3 to 6 take the linear Poisson problem through its weak form, its optimal-control formulation, the adjoint derivation of the gradient, and numerical verification, and Sections 7 to 9 repeat the same programme for the nonlinear Burgers problem. Section 10 reproduces the mesh-dependence experiment, Section 12 develops the two-level preconditioning contribution, and Section 11 traces, at the level of source code, exactly where the inner product enters each limited-memory optimiser. Section 13 concludes.

1.1 Contributions

The novel contributions of this dissertation are the following.

1. A systematic determination, across a mesh-refinement sweep spanning ratios from four to 128, of which of the optimisation algorithms in the current PETSc TAO library honour the function-space inner product and which do not.
2. Verification that TAO's limited-memory quasi-Newton method honours the inner product through its initial-Hessian seed, and is mesh-independent once its limited-memory history is large enough; this resolves an apparent failure to reproduce the reference results of Schwedes et al. [13] that had earlier been attributed to an internal Euclidean inner product.
3. A demonstration that the Newton line-search method is mesh-independent in its outer iterations while the mesh-dependence reappears in its inner Krylov solve, and that preconditioning that inner solve with the Riesz map removes it, reducing the inner iteration count from over a thousand to a constant across the mesh sweep.
4. The principal contribution of this work: a production-quality implementation of that inner preconditioner, `RieszMapPC`. Realised as a PETSc Python preconditioner, it applies the Riesz map of each pyadjoint `Control` to the reduced-Hessian Krylov solve inside TAO's Newton method, reading the metric directly from the `Control` so that a user obtains a mesh-independent inner solve simply by attaching a Riesz map to the control, with none of the manual plumbing of the research prototype. Together with a Firedrake regression test that exercises it end to end, it has been prepared and submitted as pull requests to the upstream pyadjoint and Firedrake libraries, so that the two-level fix is available to every user of the stack rather than confined to this dissertation. To our knowledge it is the first implementation to expose and remove this inner-level mesh-dependence in an adjoint optimisation pipeline.
5. A diagnosis of the two remaining mesh-dependent algorithms: the bounded quasi-Newton line-search method, which is curable by supplying the same seed by hand, and nonlinear conjugate gradient, which is shown to be structurally incurable within TAO for want of any interface at which the metric could be supplied.

2 Mathematical Preliminaries

Before we can derive the adjoint equation, discretise the optimisation problem, or talk about what "mesh-independent" even means, we need to establish some background, from the equations themselves to the functional-analytic objects that the mesh-dependence story rests on. Here we loosely follow the first chapter of Schwedes et al. [13].

2.1 Partial differential equations

First, we need the definition of a PDE. A PDE (partial differential equation) is an equation that relates an unknown function of several variables to its partial derivatives. For example we can have:

- The heat equation $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$, which describes how a temperature $u(x,t)$ spreads along a rod over time.
- The wave equation $\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$, which describes the displacement $u(x,t)$ of a vibrating string.

The two main model problems in this dissertation are Poisson's equation $-\Delta u = m$, where Δ is the Laplacian (the sum of the second partial derivatives), and the viscous Burgers equation. Poisson's equation has no time dependence and so describes a steady state, while the Burgers equation is nonlinear and time-dependent.

2.2 The PDE-constrained optimisation problem

We are interested in problems where we want to recover an unknown input to a physical system from indirect observations of its output. So, it is useful for us to fix the terminology that we will use to describe this class of problems. Every problem we study is built from the following four components:

The first is the *state variable* u. This is the quantity that describes the response of the physical system.

The second is the *control variable* m. This is the input that we are free to vary, such as a source term, a boundary value, or a coefficient. The choice of control is what distinguishes one instance of the system from another, and this is the variable that we want to optimise over.

The third is the *state equation, the PDE that governs the system and links the state to the control.* For each admissible control m, the state equation determines a corresponding state u. In this sense, the state is a function of the control (we make this dependence precise later using the solution operator of Definition 4.3).

The final component is the *objective functional* (or cost functional) $J(u,m)$, which measures how good a given state and control pair is. It usually penalises the mismatch between the state u and a prescribed desired state d, together with the size of the control. A *PDE-constrained optimisation problem* is then the problem of choosing the control m that minimises $J(u,m)$, subject to the constraint that u and m satisfy the state equation.

2.3 Domain and function spaces

Since a PDE is an equation about a function on a domain, we must first define both functions and domains properly. The classical setting (where we have that solutions are smooth enough to differentiate pointwise everywhere) is too restrictive for the elliptic problems of this dissertation (a class of steady, time-independent PDEs that includes Poisson's equation). Their natural solutions only have derivatives in a weak sense, where derivatives are defined by doing integration against test functions[1]. So, we need a function space which is large enough to contain them. The Sobolev spaces $H^1(\Omega)$ and $H^1_0(\Omega)$ give us exactly this [5, §5.2].

[1] A test function here is just a smooth function that we are free to choose, and the weak derivative is defined by asking the usual integration-by-parts formula to hold against every such test function, rather than by differentiating at each point. We show exactly how this works for Poisson's equation in Section 3.2; for now it is enough to know that it lets us make sense of derivatives for functions that are not smooth

Let $\Omega \subset \mathbb{R}^d$, with $d \in \{1, 2, 3\}$, be a bounded open domain with Lipschitz boundary $\partial\Omega$.[2] We assume that all functions are defined on Ω, unless stated otherwise. We denote by $L^2(\Omega)$ the space of square-integrable functions on Ω, equipped with the inner product and norm

$$(u,\, v)_{L^2(\Omega)} := \int_\Omega uv \,\mathrm{d}x, \qquad \|u\|_{L^2(\Omega)} := (u,\, u)^{1/2}_{L^2(\Omega)}. \tag{1}$$

Definition 2.1 (Sobolev space $H^1(\Omega)$)**.** The *Sobolev space* $H^1(\Omega)$ is defined by

$$H^1(\Omega) := \left\{ u \in L^2(\Omega) \;\middle|\; \nabla u \in (L^2(\Omega))^d \right\}. \tag{2}$$

So, we have that a function u belongs to $H^1(\Omega)$ if both the function itself and its first (weak) derivatives are square-integrable. This space is equipped with the norm

$$\|u\|^2_{H^1(\Omega)} := \|u\|^2_{L^2(\Omega)} + \|\nabla u\|^2_{L^2(\Omega)}. \tag{3}$$

This norm comes from the H^1 inner product,

$$(u,\, v)_{H^1(\Omega)} := (u,\, v)_{L^2(\Omega)} + (\nabla u,\, \nabla v)_{L^2(\Omega)} = \int_\Omega uv \,\mathrm{d}x + \int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x. \tag{4}$$

To also incorporate homogeneous Dirichlet boundary conditions into the function space itself, we also want to introduce the subspace $H^1_0(\Omega)$.

Definition 2.2 (Sobolev space $H^1_0(\Omega)$)**.** The *Sobolev space* $H^1_0(\Omega)$ is defined as the closure of $C^\infty_c(\Omega)$ in the $H^1(\Omega)$ norm.

Here, $C^\infty_c(\Omega)$ denotes the smooth functions with compact support in Ω (i.e., smooth functions which vanish in a neighbourhood of $\partial\Omega$). Taking the *closure* of this set in the H^1 norm means we also include every function that can be approximated arbitrarily well, in the H^1 norm, by such smooth functions. Functions in $H^1_0(\Omega)$ are zero on the boundary $\partial\Omega$, which makes this the natural solution space for problems with homogeneous Dirichlet boundary conditions.

They also satisfy the following inequality, which we can use to show well-posedness.

Theorem 2.3 (Poincaré inequality)**.** *There is a constant $C_\Omega > 0$, depending only on Ω, such that $\|u\|_{L^2(\Omega)} \leq C_\Omega \|\nabla u\|_{L^2(\Omega)}$ for all $u \in H^1_0(\Omega)$ [1].*

The spaces $L^2(\Omega)$, $H^1(\Omega)$ and $H^1_0(\Omega)$ defined above are all examples of *Hilbert spaces*, and the mesh-dependence story depends on the choice of inner product that is placed on such a space. We record the two underlying notions precisely.

Definition 2.4 (Inner product)**.** An *inner product* on a real vector space H is a map $(\cdot, \cdot) : H \times H \to \mathbb{R}$. More precisely, let $u, v, w \in H$ be vectors and $\alpha \in \mathbb{R}$ a scalar; then it satisfies the following four properties:

1. $(u + v, w) = (u, w) + (v, w)$ (additivity)
2. $(\alpha v, w) = \alpha\,(v, w)$ (homogeneity)
3. $(v, w) = (w, v)$ (symmetry)

[2] Informally, a Lipschitz boundary is one that is not too rough: corners like a polygon's are allowed, but not cusps or fractal-like features. The unit square used in the experiments qualifies.

4. $(v, v) \geq 0$, with equality if and only if $v = 0$ (positive definiteness)

An inner product induces the norm $\|v\| := (v, v)^{1/2}$.

Definition 2.5 (Hilbert space)**.** A *Hilbert space* is an inner product space that is *complete*, meaning every Cauchy sequence has a limit in the space.

Throughout the dissertation, the control variable m lives in a Hilbert space M, the control space, equipped with an inner product $(\cdot, \cdot)_M$ and associated norm $\|\cdot\|_M$.

2.4 Dual spaces and the Riesz representation theorem

As we see later in this dissertation, the mesh-dependence story here is, at its root, a story about a choice of inner product. For an inner product to be the subject of a choice, we first need to understand why we have any freedom to choose this at all. This requires understanding the relationship between a Hilbert space and its dual. The derivative of a real-valued functional on a Hilbert space is naturally an element of the dual space (a linear functional), but actually in practice we want to picture it as an element of the original space ("a gradient"). So, we introduce the Riesz representation theorem which provides a canonical bijection between the two views. This bijection depends on the choice of inner product, and that dependence is exactly what the mesh-dependence study in Section 10 depends on. Here we set up the statements that we need, and the consequences are described in Section 10.

Let H be a Hilbert space with inner product $(\cdot, \cdot)_H$.

Definition 2.6 (Dual space)**.** The *dual space* of H, denoted H^*, is the space of all continuous linear functionals on H [13, p. 21]:

$$H^* := \big\{ f : H \to \mathbb{R} \;\big|\; f \text{ is linear and continuous} \big\}. \tag{5}$$

We write duality pairings between H^* and H in angle brackets, $\langle f, v \rangle$, and inner products in H in parentheses, $(u, v)_H$, so that the bracket style can tell the reader which kind of object lives on each side. One specific case that we will use throughout the dissertation is the Sobolev dual $H^{-1}(\Omega) := H_0^1(\Omega)^*$.

Although H and H^* are different spaces of different objects, the Riesz representation theorem below gives a canonical bijection between them [13, Thm 1.1].

Theorem 2.7 (Riesz representation theorem)**.** *Let H be a Hilbert space with inner product $(\cdot, \cdot)_H$. For any continuous linear functional $f \in H^*$, there exists a unique $u_f \in H$ such that*

$$f(v) = (u_f, v)_H \qquad \forall v \in H, \tag{6}$$

and the norms agree, i.e. $\|f\|_{H^*} = \|u_f\|_H$.

This bijection is what lets us pass between the dual view and the primal view of a linear functional, and the map that effects the bijection is called the Riesz map:

Definition 2.8 (Riesz map and Riesz representer)**.** The map $\mathcal{R}_H : H^* \to H$ that sends each $f \in H^*$ to its unique representative $u_f \in H$ is called the *Riesz map*. The element $\mathcal{R}_H(f) := u_f$ is called the *Riesz representer* of f in H.

The Riesz map depends on the choice of inner product on H: the same linear functional has different Riesz representers under different inner products.

2.5 Differential calculus in function spaces

To minimise a functional, we have to differentiate it, and the adjoint method in particular depends on the chain rule applied to a composite operator. In finite dimensions, partial derivatives stacked into a vector are enough. However, the optimisation variable here lives in an infinite-dimensional Hilbert space, so we even have to be careful about what "direction" actually means. So, we define the Fréchet derivative as the right replacement for the finite-dimensional gradient: a bounded linear operator[3] that captures the first-order change of a map between Hilbert spaces [13, §1.2.2–1.2.3].

Let U and V be Hilbert spaces, and let $F : U \to V$ be a (possibly nonlinear) operator.

Definition 2.9 (Fréchet derivative)**.** The operator F is *Fréchet differentiable* at $u \in U$ if there exists a bounded linear operator $\mathrm{d}F(u) \in \mathcal{L}(U, V)$, where $\mathcal{L}(U, V)$ denotes the space of bounded linear operators from U to V, such that

$$\lim_{\|\delta u\|_U \to 0} \frac{\|F(u + \delta u) - F(u) - \mathrm{d}F(u)\,\delta u\|_V}{\|\delta u\|_U} = 0. \tag{7}$$

The directional derivative in the direction $\delta u \in U$ is written $\mathrm{d}F(u; \delta u) := \mathrm{d}F(u)\,\delta u$.

When an operator or functional depends on multiple variables, we can also naturally extend the idea of Fréchet differentiation to partial derivatives.

Definition 2.10 (Partial Fréchet derivative)**.** Let $F : U \times M \to V$. The *partial Fréchet derivative* of F with respect to u at the point (u, m), acting in the direction $\delta u \in U$, is denoted as $\partial_u F(u, m; \delta u) \in V$. It is defined as the Fréchet derivative of the map $u \mapsto F(u, m)$ with fixed m. The partial derivative with respect to m, denoted $\partial_m F(u, m; \delta m)$, is defined analogously.

Finally, we can get the derivative of a composite functional from the chain rule.

Theorem 2.11 (Chain rule)**.** *Let M and U be Hilbert spaces. Let $S : M \to U$ be Fréchet differentiable at $m \in M$, and let $J : U \times M \to \mathbb{R}$ be continuously Fréchet differentiable at $(S(m), m)$. Then the reduced functional $\tilde{J}(m) := J(S(m), m)$ is Fréchet differentiable at m, and its derivative in the direction $\delta m \in M$ is given by*

$$\mathrm{d}\tilde{J}(m;\, \delta m) = \partial_u J\big(S(m), m;\, \mathrm{d}S(m;\, \delta m)\big) + \partial_m J\big(S(m), m;\, \delta m\big). \tag{8}$$

2.6 Bilinear forms and weak formulations

Even with the function spaces and calculus in place, the PDEs we want to solve are still written in their classical, pointwise form, but this form requires too much smoothness from the solution. The standard fix is to multiply by a test function and integrate by parts, moving derivatives off the solution and onto the test function. The reformulated equation (called the *weak* or *variational* form) is exactly what the finite element method discretises, and whether it exists or not is determined by the bilinear form that comes from it. The Lax-Milgram theorem turns two simple conditions on that bilinear form (continuity and coercivity) into existence and uniqueness of a weak solution. This transfer of derivatives is useful as it lets us look for solutions in Sobolev spaces instead of in spaces of smooth functions.

Let V be a Hilbert space. A bilinear form $a(\cdot, \cdot) : V \times V \to \mathbb{R}$ is *continuous* if there exists a constant $C > 0$ such that

$$|a(u, v)| \le C\|u\|_V \|v\|_V \qquad \forall u, v \in V, \tag{9}$$

[3] A linear map between the two spaces whose output norm is at most a fixed multiple of the input norm.

and *coercive* if there exists a constant $\alpha > 0$ such that

$$a(u, u) \geq \alpha \|u\|_V^2 \qquad \forall u \in V. \tag{10}$$

These two properties are the hypotheses of the following Lax-Milgram theorem [5, Thm 4.36].

Theorem 2.12 (Lax-Milgram)**.** *Let V be a Hilbert space and let $a(\cdot, \cdot) : V \times V \to \mathbb{R}$ be a continuous, coercive bilinear form. Then for every continuous linear functional $\ell \in V^*$ there exists a unique $u \in V$ such that*

$$a(u, v) = \ell(v) \qquad \forall v \in V. \tag{11}$$

We use this existence and uniqueness later, where it makes the solution operator $S : m \mapsto u$ of Definition 4.3 well-defined, and hence the reduced functional we minimise.

3 The Forward Poisson Problem

With the function spaces, weak formulations, and the Lax-Milgram theorem now in place, we are ready to study the first of our two model problems, the Poisson equation. We look at this first as it is a relatively simple problem: it is linear, and elliptic, with no time dependence. This makes it the natural setting in which we can build up a PDE-constrained optimisation problem (the forward solve, the adjoint equation, the reduced gradient, and the numerical optimisation) one piece at a time, before we repeat the same programme for the harder, nonlinear Burgers equation in Section 7. It is also natural to begin here because the existence and uniqueness of a solution follow directly from the Lax-Milgram theorem we have just stated. In this section we take the first step, the *forward* problem: for a given control (here the source term m) we solve for the corresponding state u, leaving the optimisation over m to Section 4.

Our goal is to rewrite the PDE in a form that involves only first derivatives of the solution, so that we can look for solutions in $H_0^1(\Omega)$ rather than in spaces of classical (twice differentiable) solutions.

3.1 Strong formulation

Let m be a given source term (for example $m \in L^2(\Omega)$). The strong form of the forward problem is

$$-\Delta u = m \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega. \tag{12}$$

3.2 Derivation of the weak form

We choose a test function in

$$v \in H_0^1(\Omega), \tag{13}$$

which vanishes on $\partial\Omega$ in the trace sense. Multiplying equation (12) by v and integrating over Ω gives us

$$\int_\Omega (-\Delta u)\, v \,\mathrm{d}x = \int_\Omega m\, v \,\mathrm{d}x. \tag{14}$$

The left-hand side contains second derivatives of u. To remove one of them, we recall Green's first identity [5, Lemma 5.23]:

$$\int_\Omega (-\Delta u)\, v \,\mathrm{d}x = \int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x - \int_{\partial\Omega} \frac{\partial u}{\partial n}\, v \,\mathrm{d}s, \tag{15}$$

where n denotes the outward unit normal on $\partial\Omega$ and $\partial u/\partial n := \nabla u \cdot n$ is the normal derivative of u. Because $v \in H_0^1(\Omega)$ vanishes on $\partial\Omega$ (because of the boundary conditions), the boundary integral vanishes, and we obtain

$$\int_\Omega (-\Delta u)\, v \,\mathrm{d}x = \int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x. \tag{16}$$

Substituting this into equation (14) gives

$$\int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x = \int_\Omega m\, v \,\mathrm{d}x, \tag{17}$$

and this identity must hold for all test functions $v \in H_0^1(\Omega)$. We collect the result into the following definition.

Definition 3.1 (Weak formulation of the forward Poisson problem)**.** Let $m \in L^2(\Omega)$. The *weak formulation* of equation (12) is: find $u \in H_0^1(\Omega)$ such that

$$\int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x = \int_\Omega m\, v \,\mathrm{d}x \qquad \forall v \in H_0^1(\Omega). \tag{18}$$

It is also convenient to introduce the bilinear form

$$b(u, v) := \int_\Omega \nabla u \cdot \nabla v \,\mathrm{d}x \tag{19}$$

and the linear functional

$$\ell(v) := \int_\Omega m\, v \,\mathrm{d}x, \tag{20}$$

so that equation (18) now reads

$$\text{find } u \in H_0^1(\Omega) \text{ such that } b(u, v) = \ell(v) \quad \forall v \in H_0^1(\Omega). \tag{21}$$

The bilinear form b is continuous and, by the Poincaré inequality, coercive on $H_0^1(\Omega)$. The linear functional ℓ is bounded on $H_0^1(\Omega)$ whenever $m \in L^2(\Omega)$. The Lax-Milgram theorem then guarantees existence and uniqueness of the weak solution.

4 The Poisson Optimal Control Problem

Now, we have set up the forward problem and showed it is well posed, so each source term m determines the state u. We now reverse the question. Rather than taking the source as given, we ask which source term m makes the resulting state u best match a given desired state d, while penalising large controls. In practice it is often this reversed direction, achieving a desired outcome rather than predicting one, that we care about. As an example, if we let u be the steady-state temperature of a body whose boundary is held fixed and m be a distributed heat source, we are asking which heating m brings the temperature closest to a desired profile d without expending excessive power. Problems of this shape, in which an input to a PDE is tuned to steer its solution towards a target, are very common in inverse problems and optimal design. The Poisson optimal control problem is a PDE-constrained optimisation problem of the form introduced in Section 2.2 [13].

Throughout the Poisson sections, we set

$$V := H_0^1(\Omega), \qquad M := L^2(\Omega). \tag{22}$$

4.1 Objective functional

Let $d \in L^2(\Omega)$ be a given desired state and let $\alpha > 0$.

Definition 4.1 (Poisson objective functional)**.** The *objective functional* $J : V \times M \to \mathbb{R}$ is defined by

$$J(u, m) := \tfrac{1}{2}\|u - d\|^2_{L^2(\Omega)} + \tfrac{\alpha}{2}\|m\|^2_M. \tag{23}$$

The first term measures the misfit between the state u and the desired state d. The second term penalises large controls; the parameter $\alpha > 0$ balances accuracy of fit against regularity of the control. Note that without the regularisation term the problem would be ill posed: the control-to-state map is smoothing, so many very different controls would produce nearly the same state and the misfit term alone fixes no unique, stable minimiser. The quadratic penalty makes the objective strictly convex and coercive in m, which ensures a unique minimiser that depends continuously on the data.

4.2 The constrained optimisation problem

Combining Definition 4.1 with the weak state equation equation (21), we get that the PDE-constrained optimisation problem reads as follows.

Definition 4.2 (Poisson PDE-constrained optimisation problem)**.** The *Poisson PDE-constrained optimisation problem* is to find $(u, m) \in V \times M$ minimising

$$\min_{(u,m)\in V\times M} J(u, m) \tag{24}$$

subject to

$$b(u, v) = \ell(v) \qquad \forall v \in V, \tag{25}$$

where the bilinear form b and the linear functional ℓ are those of equations (19) and (20).

4.3 Reduced formulation

The problem of Definition 4.2 is a *constrained* minimisation: u and m are not independent, but are instead tied together by the state equation, and such problems in $V \times M$ are awkward to attack with the gradient-based methods that we will use in the rest of this dissertation. However, since the state equation determines u uniquely from m, we have that the state is not a free unknown, and we can remove it, reducing the problem to an *unconstrained* minimisation over the control alone [13].

For each $m \in M$ there exists a unique state $u \in V$ satisfying the state equation, so we may view the state as a function of the control. To make this precise, we name the map that sends each control to its state as the solution operator:

Definition 4.3 (Poisson solution operator)**.** The *solution operator* $S : M \to V$ assigns to each $m \in M$ the unique solution $S(m) := u(m) \in V$ of the weak state equation associated with m.

Using the solution operator we eliminate the state from the optimisation problem.

Definition 4.4 (Poisson reduced functional)**.** The *reduced functional* $\tilde{J} : M \to \mathbb{R}$ is defined by

$$\tilde{J}(m) := J(S(m), m) = J(u(m), m). \tag{26}$$

The second equality is only a change of notation: $S(m)$ and $u(m)$ are two names for the same object, the unique state produced by the control m. The key step is that $\tilde{J}$ is a function of m alone, which is legitimate because the state equation determines that state uniquely, so that once m is fixed no independent choice of u remains.

We then have that the constrained problem of Definition 4.2 is equivalent to the unconstrained minimisation problem

$$\min_{m \in M} \tilde{J}(m). \tag{27}$$

This reduced functional is the central object of the rest of this work: it is what every optimiser we study minimises, its gradient (computed by the adjoint method of Section 5) is what the Riesz map acts on, and its Hessian is where the mesh-dependence of Section 10 appears.

5 Derivative of the Reduced Functional and the Adjoint Equation

We have now reduced the problem to the unconstrained minimisation of $\tilde{J}$ over the control space. We want to do this minimisation with a gradient-based method, the natural choice for optimising over a function space, but these methods need the derivative of $\tilde{J}$. Getting it by differentiating through the state equation would require a separate linearised solve for each control direction δm, whereas if we use the adjoint method, we can get the derivative in every direction from a single additional solve. It does this by introducing an auxiliary variable, the *adjoint*, whose governing equation is chosen precisely so that the unknown state variation cancels from the expression for the derivative; the derivative in every direction can then be obtained from one adjoint solution. We now carry out this derivation for the Poisson problem, using the chain rule (Theorem 2.11) and the calculus of partial Fréchet derivatives (Definition 2.10) introduced in Section 2.5.

Applying the chain rule to $\tilde{J}(m) = J(u(m), m)$ in the direction $\delta m \in M$ gives

$$\mathrm{d}\tilde{J}(m;\, \delta m) = \partial_u J(u, m;\, \mathrm{d}u_m(m;\, \delta m)) + \partial_m J(u, m;\, \delta m), \tag{28}$$

where $\mathrm{d}u_m(m;\, \delta m)$ is the total derivative of the state with respect to the control in the direction δm [13, p. 32]. Writing

$$\delta u := \mathrm{d}u_m(m;\, \delta m) \in V, \tag{29}$$

this becomes

$$\mathrm{d}\tilde{J}(m;\, \delta m) = \partial_u J(u, m;\, \delta u) + \partial_m J(u, m;\, \delta m). \tag{30}$$

Evaluating this derivative requires the partial derivatives of two different objects, which we compute separately in the next two subsections. The state variation δu is not a free quantity: it is fixed by the state equation, so determining it requires the partial derivatives of the forward model operator F (the constraint), which we treat first in Section 5.1. The two remaining terms are partial derivatives of the objective functional J (the cost), which we compute in Section 5.2.

5.1 Tangent linear equation

To compute δu we differentiate the state equation. We define a *forward model operator* that encodes the residual of the weak problem.

Definition 5.1 (Forward model operator)**.** The *forward model operator* is defined as the residual of the weak state equation:

$$F(u, m;\, v) := b(u, v) - (m,\, v)_{L^2(\Omega)}. \tag{31}$$

The weak state equation can then be written as $F(u(m), m;\, v) = 0$ for all $v \in V$ [13, p. 29]. Differentiating with respect to m, the chain rule gives us

$$\partial_u F(u, m;\, \delta u, v) + \partial_m F(u, m;\, \delta m, v) = 0 \qquad \forall v \in V. \tag{32}$$

We compute the two partial derivatives of the forward model operator F. First, we want the derivative with respect to u. For $\varepsilon \in \mathbb{R}$, set $u_\varepsilon := u + \varepsilon \delta u$. Using linearity of b in its first argument,

$$F(u_\varepsilon, m;\, v) = b(u, v) - (m,\, v)_{L^2(\Omega)} + \varepsilon\, b(\delta u, v). \tag{33}$$

The corresponding difference quotient is

$$\frac{F(u + \varepsilon\delta u, m;\, v) - F(u, m;\, v)}{\varepsilon} = b(\delta u, v), \tag{34}$$

and taking $\varepsilon \to 0$ gives

$$\partial_u F(u, m;\, \delta u, v) = b(\delta u, v). \tag{35}$$

Next, the derivative with respect to m. By the analogous argument, perturbing m in the direction δm, we get

$$\partial_m F(u, m;\, \delta m, v) = -(\delta m,\, v)_{L^2(\Omega)}. \tag{36}$$

Combining equations (35) and (36) with equation (32) gives the *tangent linear equation*, also known as the tangent linear model [13, p. 32].

Proposition 5.2 (Tangent linear equation)**.** *The state variation $\delta u \in V$ satisfies*

$$b(\delta u, v) = (\delta m,\, v)_{L^2(\Omega)} \qquad \forall v \in V. \tag{37}$$

5.2 Partial derivatives of the objective functional

Now we compute the partial derivatives of the the objective functional J. First, the derivative of J with respect to u. Setting $u_\varepsilon := u + \varepsilon v$ and expanding, we get

$$J(u_\varepsilon, m) = J(u, m) + \varepsilon(u - d,\, v)_{L^2(\Omega)} + \tfrac{\varepsilon^2}{2}\|v\|^2_{L^2(\Omega)}, \tag{38}$$

and so

$$\partial_u J(u, m;\, v) = (u - d,\, v)_{L^2(\Omega)}. \tag{39}$$

Next, the derivative of J with respect to m. Setting $m_\varepsilon := m + \varepsilon\delta m$ and expanding $\|m + \varepsilon\delta m\|^2$,

$$J(u, m_\varepsilon) = J(u, m) + \alpha\varepsilon(m,\, \delta m)_{L^2(\Omega)} + \tfrac{\alpha\varepsilon^2}{2}\|\delta m\|^2_{L^2(\Omega)}, \tag{40}$$

giving

$$\partial_m J(u, m;\, \delta m) = \alpha(m,\, \delta m)_{L^2(\Omega)}. \tag{41}$$

5.3 Adjoint problem

We now introduce the adjoint variable that carries out this cancellation [13, p. 33]. The adjoint equation reverses the direction of the state equation: where the state equation maps the control forward to the state, the adjoint equation propagates the sensitivity of the objective backward, to the control. The only obstacle to evaluating $\mathrm{d}\tilde{J}$ cheaply is the term $\partial_u J(u, m;\, \delta u)$ in the chain rule, which drags in the direction-specific state variation δu. The adjoint is defined precisely to remove it, as we carry out in Section 5.4. First we define the adjoint variable:

Definition 5.3 (Adjoint variable)**.** The *adjoint variable* $\lambda \in V$ is the unique solution of

$$\partial_u F(u, m;\, \lambda, v) = \partial_u J(u, m;\, v) \qquad \forall v \in V. \tag{42}$$

Substituting equations (35) and (39) into the definition gives the adjoint equation for the Poisson problem in concrete form.

Proposition 5.4 (Adjoint equation)**.** *The adjoint variable $\lambda \in V$ satisfies*

$$b(\lambda, v) = (u - d,\, v)_{L^2(\Omega)} \qquad \forall v \in V. \tag{43}$$

The *adjoint problem* is to find the $\lambda \in V$ that satisfies this adjoint equation. It is a single Poisson-type solve: the same bilinear form b as the forward problem of Definition 3.1, but driven by the misfit $u - d$ in place of the control m. The bilinear form b is continuous and coercive on V, and the right-hand side is a continuous linear functional on V, so existence and uniqueness of the adjoint follow from Lax-Milgram [13, p. 14]. Its one solution λ is all we need to evaluate the derivative of $\tilde{J}$ in every direction, which is what we do next.

5.4 Derivative of the reduced functional

With the objective partials and the adjoint now, we can get the derivative of $\tilde{J}$ and eliminate the state variation δu. Substituting equations (39) and (41) into the chain-rule expression equation (30) gives

$$\mathrm{d}\tilde{J}(m;\, \delta m) = (u - d,\, \delta u)_{L^2(\Omega)} + \alpha(m,\, \delta m)_{L^2(\Omega)}. \tag{44}$$

We now eliminate δu using the adjoint and tangent linear equations. Choosing $v = \delta u$ in equation (43) and using the symmetry of b (as we have that $b(u, v) = b(v, u)$ since $\nabla u \cdot \nabla v = \nabla v \cdot \nabla u$),

$$(u - d,\, \delta u)_{L^2(\Omega)} = b(\delta u, \lambda). \tag{45}$$

Choosing $v = \lambda$ in equation (37), we get that

$$b(\delta u, \lambda) = (\delta m,\, \lambda)_{L^2(\Omega)}, \tag{46}$$

and so

$$\mathrm{d}\tilde{J}(m;\, \delta m) = (\lambda,\, \delta m)_{L^2(\Omega)} + \alpha(m,\, \delta m)_{L^2(\Omega)} \qquad \forall \delta m \in M, \tag{47}$$

where $u = u(m)$ is the state corresponding to m and λ is the adjoint solution.

5.5 The L^2-gradient

We now have the derivative $\mathrm{d}\tilde{J}$, but a gradient-based method cannot use it as it stands. These methods improve the control by stepping along a search direction, so it needs an actual element of the control space M to add to the current control m, whereas the derivative is a functional that acts on directions rather than being one itself. So, we need to convert it into a genuine direction in M, and it is in this conversion that the choice of inner product matters.

The reduced derivative equation (47) is a continuous linear functional acting on directions $\delta m \in M$, which means that it is an element of the dual space M^*. By the Riesz representation theorem (Theorem 2.7), there exists a unique element of M representing it.

Definition 5.5 (L^2-gradient)**.** Let $\mathcal{R}_{L^2} : M^* \to M$ denote the Riesz map associated with the $L^2(\Omega)$ inner product. The *L^2-gradient* of $\tilde{J}$ at m is defined by

$$\nabla \tilde{J}(m) := \mathcal{R}_{L^2}(\mathrm{d}\tilde{J}(m)) \in M, \tag{48}$$

that is, $\nabla \tilde{J}(m)$ is the unique element of M satisfying

$$\mathrm{d}\tilde{J}(m;\, \delta m) = (\nabla \tilde{J}(m),\, \delta m)_{L^2(\Omega)} \qquad \forall \delta m \in M. \tag{49}$$

Linearity of the inner product lets us factor equation (47) as

$$\mathrm{d}\tilde{J}(m;\,\delta m) = (\lambda + \alpha m,\,\delta m)_{L^2(\Omega)} \qquad \forall \delta m \in M. \tag{50}$$

Comparing with the definition of $\nabla \tilde{J}$ gives us that

$$\nabla \tilde{J}(m) = \lambda + \alpha m. \tag{51}$$

This completes the gradient derivation for the Poisson problem: a single forward solve for the state u and a single adjoint solve for λ combine to give $\nabla \tilde{J}(m) = \lambda + \alpha m$, the direction a gradient-based optimiser consumes at each step.

6 Numerical Implementation and Verification of the Poisson Problem

Having derived the weak form of the forward problem, the adjoint equation and the L^2-gradient of the reduced functional, we now implement the Poisson optimal control problem numerically. The implementation serves two purposes: first, to verify that the finite element discretisation of the forward problem behaves as expected; second, to confirm that the adjoint-based gradient computation is correct before extending the same approach to more complicated problems in Sections 9 and 10.

We carry out the numerical work in two stages. First, we implement the forward Poisson problem in plain Python using a simple P_1 finite element method on a triangular mesh; the hand-written implementation makes the main finite element steps explicit, namely mesh construction, basis functions, element-wise assembly and the imposition of homogeneous Dirichlet boundary conditions. Second, we implement the same forward problem in Firedrake [11] together with the adjoint solve, the gradient computation and the Taylor test. The two implementations share the same mathematical content; the first makes the finite element method explicit, and the second expresses it more compactly using a domain-specific language.

6.1 Forward Poisson by hand

We start from scratch, without a finite element library, so that nothing about the method is hidden. Implementing the forward solve by hand puts every step, the mesh, the basis functions, the assembly and the boundary conditions, in plain view, where each can be checked directly against the weak form of Section 3.

We implement the forward problem in plain Python using a continuous piecewise linear finite element method. The domain $\Omega = (0, 1)^2$ is triangulated using a structured mesh obtained by splitting each square cell into two triangles (Figure 1), and the discrete space is the standard P_1 Lagrange space.

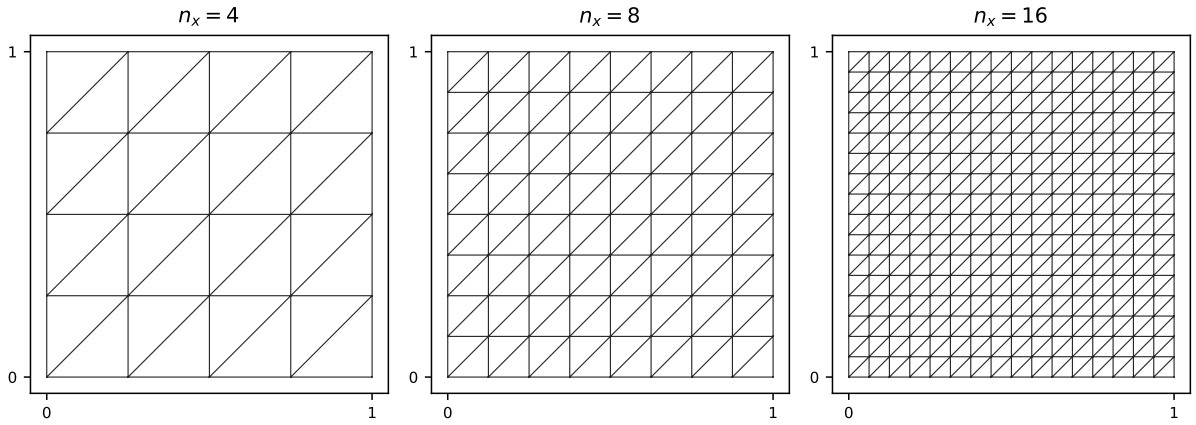

Figure 1: The structured triangulation of the unit square $\Omega = (0,1)^2$ we used for the forward Poisson problem, shown at three refinement levels $n_x = 4, 8, 16$. Each square cell is split into two triangles along its bottom-left to top-right diagonal, giving $2n_x^2$ elements and $(n_x + 1)^2$ vertices. The convergence study refines this mesh uniformly up to $n_x = 512$. The hand-written implementation and Firedrake's `UnitSquareMesh` produce this identical mesh.

First we define two finite element notions that the assembly below relies on: the degrees of freedom it solves for, and the numerical quadrature it uses to evaluate the element integrals.

Definition 6.1 (Degree of freedom)**.** A *degree of freedom* is one of the unknowns in the finite element system: the coefficient multiplying a single basis function. For the P_1 Lagrange space used here, there is one degree of freedom per mesh vertex.

Definition 6.2 (Numerical quadrature)**.** A *quadrature rule* approximates the integral of a function f over an element K by a weighted sum of its values at finitely many points,

$$\int_K f \,\mathrm{d}x \approx \sum_q w_q \, f(x_q), \tag{52}$$

where the x_q are the *quadrature points* and the w_q the corresponding *weights*. The rule has *degree* p if this approximation is exact whenever f is a polynomial of degree at most p.

We assemble the global linear system one element at a time. For each triangle, we first map it onto a reference triangle and apply numerical quadrature (Definition 6.2), which gives us a local stiffness matrix and a local load vector. We then add these local contributions into a single global sparse system. Finally, since the problem has homogeneous Dirichlet boundary conditions, we modify the assembled system so that the boundary degrees of freedom (Definition 6.1) are fixed to zero. These steps are summarised in Algorithm 1, and the full implementation is available in the accompanying code repository.

Algorithm 1 Assembly and solution of the P_1 forward Poisson system, as carried out by hand.

construct the structured triangular mesh of Ω
initialise the global stiffness matrix $A \leftarrow 0$ and load vector $b \leftarrow 0$
for all triangles K with vertices x_0, x_1, x_2 **do**
    form the affine map to the reference triangle, $J \leftarrow [\, x_1 - x_0 \;\; x_2 - x_0 \,]$
    $\text{area}(K) \leftarrow \frac{1}{2}|\det J|$
    compute the physical basis gradients $\nabla \phi_i \leftarrow J^{-\top} \, \nabla_\xi \, \hat{\phi}_i$
    local stiffness $A^K_{ij} \leftarrow \text{area}(K) \;\; \nabla \phi_i \cdot \nabla \phi_j$
    local load $b^K_i \leftarrow \sum_q w_q \, |\det J| \, m(x_q) \, \hat{\phi}_i(\xi_q)$ ▷ quadrature
    add A^K and b^K into the global A and b
end for
for all boundary vertices p **do** ▷ homogeneous Dirichlet condition
    overwrite row p of A with the identity row and set $b_p \leftarrow 0$
end for
solve the linear system $Au = b$

The hand-written implementation focuses more on clarity rather than efficiency - writing the assembly explicitly makes the connection between the weak form derived in Section 3 and the finite element matrices that appear in the computation transparent.

6.2 Forward Poisson solve in Firedrake

The hand-written solve is transparent, but it only does the forward problem. Everything that follows, the adjoint solve, the gradient, and the mesh-dependence experiments, relies on the algorithmic differentiation that Firedrake and its pyadjoint extension provide, so we now want to implement the same forward problem there. This also gives an independent check, as we now have two different codes should agree on the same discretisation.

So, we now implement the same forward problem in Firedrake [11] using continuous Galerkin elements of degree one. The variational forms are written directly from the weak formulation equation (21), and the homogeneous Dirichlet boundary condition is imposed strongly on $\partial\Omega$. The complete forward solve is shown in Figure 2.

```
V = FunctionSpace(mesh, "CG", 1)
u = TrialFunction(V)
v = TestFunction(V)

a = inner(grad(u), grad(v)) * dx
L = m * v * dx
bc = DirichletBC(V, 0.0, "on_boundary")

u_h = Function(V)
solve(a == L, u_h, bcs=bc)
```

Figure 2: The forward Poisson solve in Firedrake. The bilinear form `a` and linear form `L` are transcribed almost verbatim from the weak formulation equation (21), with `m` the source term interpolated onto the finite element space.

The Firedrake implementation is notably more concise than the plain Python version, but represents the same finite element problem. Working in Firedrake also lets us express the forward solve, the adjoint solve and the gradient computation in a form that remains close to

the mathematical notation introduced earlier in the work; this is critical for the adjoint-based experiments of Section 10, which rely on the algorithmic differentiation provided by the pyadjoint extension to Firedrake [9].

6.3 Manufactured solution and convergence study

To test the correctness of the forward implementations, we use the method of manufactured solutions, following the presentation in Chapter 6 of Ham and Cotter [5]. We prescribe a smooth function satisfying the homogeneous boundary conditions and choose the source term accordingly. We define

$$u_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y), \tag{53}$$

which vanishes on $\partial\Omega$. The corresponding source term is

$$m(x, y) = -\Delta u_{\text{exact}}(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y). \tag{54}$$

With this choice, the continuous problem admits the known exact solution $u = u_{\text{exact}}$.

Let u_h denote the finite element approximation on a mesh of size h. For sufficiently smooth solutions, standard finite element theory predicts that the $L^2(\Omega)$ error for piecewise linear elements satisfies

$$\|u - u_h\|_{L^2(\Omega)} = \mathcal{O}(h^2). \tag{55}$$

We estimate the experimental order of convergence between two successive mesh levels by

$$q = \frac{\log(e_H/e_h)}{\log(h_H/h_h)}, \tag{56}$$

where $e_h = \|u - u_h\|_{L^2(\Omega)}$.

The errors and rates observed on the two implementations, the plain Python P_1 assembler and Firedrake on the same sequence of uniformly refined meshes, are shown side by side in Tables 1a and 1b.

n_x	h	L^2 error	rate
4	0.2500	$7.5963 \cdot 10^{-2}$	-
8	0.1250	$2.0409 \cdot 10^{-2}$	1.8961
16	0.0625	$5.2009 \cdot 10^{-3}$	1.9724
32	0.0312	$1.3066 \cdot 10^{-3}$	1.9930
64	0.0156	$3.2704 \cdot 10^{-4}$	1.9982
128	0.0078	$8.1786 \cdot 10^{-5}$	1.9996
256	0.0039	$2.0448 \cdot 10^{-5}$	1.9999
512	0.0020	$5.1121 \cdot 10^{-6}$	2.0000

(a) Hand-written P_1 implementation.

N	h	L^2 error	rate
4	0.2500	$6.2103 \cdot 10^{-2}$	-
8	0.1250	$1.8332 \cdot 10^{-2}$	1.7603
16	0.0625	$4.7854 \cdot 10^{-3}$	1.9376
32	0.0312	$1.2095 \cdot 10^{-3}$	1.9842
64	0.0156	$3.0321 \cdot 10^{-4}$	1.9960
128	0.0078	$7.5855 \cdot 10^{-5}$	1.9990
256	0.0039	$1.8967 \cdot 10^{-5}$	1.9998
512	0.0020	$4.7420 \cdot 10^{-6}$	1.9999

(b) Firedrake implementation.

Table 1: Manufactured-solution convergence tests for the forward Poisson problem, for (a) the hand-written P_1 assembler and (b) Firedrake, on the same uniformly refined meshes. In both implementations the observed rates approach the expected value of 2, giving confidence that the forward discretisation is correct.

Both sets of errors are plotted against the mesh size on log-log axes in Figure 3.

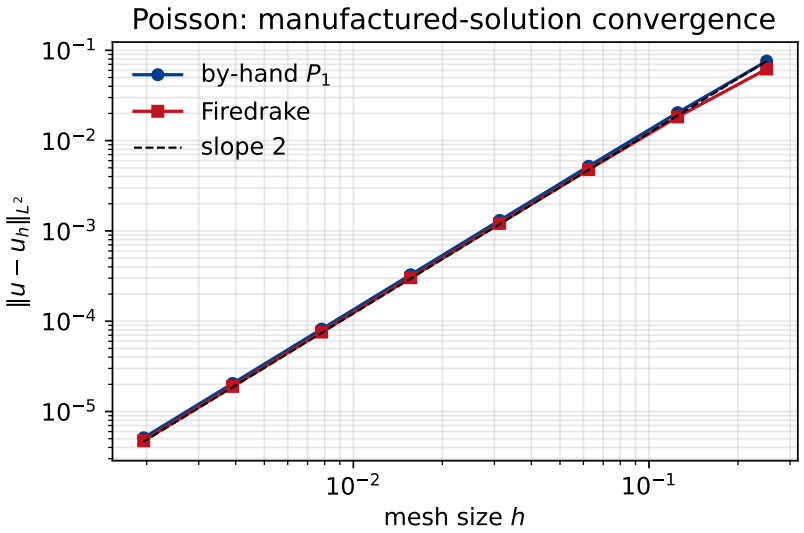

Figure 3: Manufactured-solution convergence of the two forward Poisson implementations on log-log axes (the data of Tables 1a and 1b). All three lines, the two implementations and the slope-2 reference, lie almost on top of one another.

This confirms the expected $\mathcal{O}(h^2)$ rate, since the data run parallel to the slope-2 line, but the three lines nearly coincide, so it is hard to tell the two implementations apart or to judge how closely they follow the reference. Rescaling the error by h^2 shows the same information more clearly: a second-order method then appears as a flat plateau instead of a diagonal, and the two implementations separate into distinct curves, as in Figure 4.

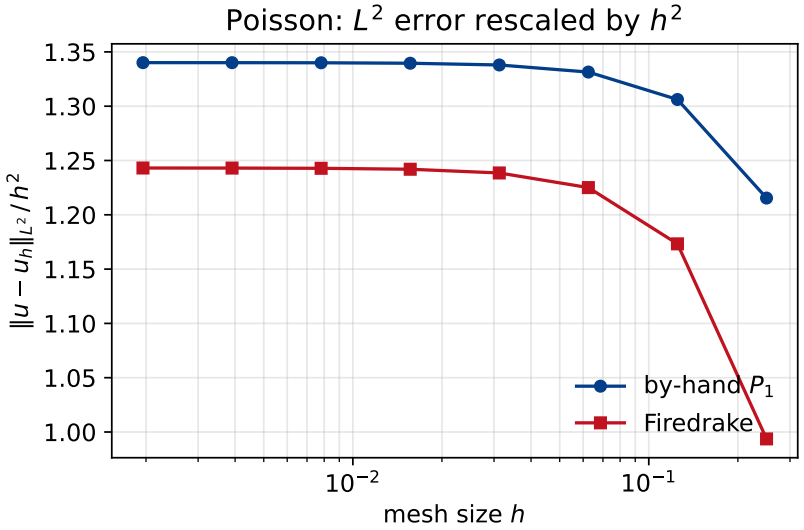

Figure 4: Manufactured-solution convergence of the two forward Poisson implementations (the data of Tables 1a and 1b), with the L^2 error rescaled by h^2. Each curve plateaus to a constant as h decreases, which is the signature of the $\mathcal{O}(h^2)$ rate for P_1 elements; the roll-off at coarse h on the right is the pre-asymptotic regime, where the rate has not yet reached two. The by-hand plateau sits a little above Firedrake's for the reason discussed below.

Both implementations recover the theoretical second-order rate, confirming that the forward discretisation is correct. The two do not report the same absolute error: the hand-written

code is consistently a little larger, and its rescaled error in Figure 4 plateaus near 1.34 against Firedrake's 1.24. This gap is a quadrature effect, not a difference in the discretisation, since both solve the same P_1 problem on the same mesh. The hand-written assembler evaluates the load vector and the L^2 error with a fixed degree-two quadrature rule, whereas Firedrake selects a higher quadrature degree for the smooth manufactured source $2\pi^2 \sin(\pi x)\sin(\pi y)$ and for the error integrand - the coarser rule integrates these slightly less accurately and so reports a marginally larger error. A degree-two rule is still exact enough to preserve the $\mathcal{O}(h^2)$ rate, which is why the two curves are parallel rather than divergent.

6.4 Adjoint solve and gradient computation

We now implement the gradient in Firedrake. This is the step that matters most for the rest of the work, as every optimiser we will later study uses this gradient at each iteration, so it must be computed correctly.

The derivation of Section 5 states that the gradient of the reduced functional is computed by first solving the adjoint equation and then forming the L^2-gradient $\nabla \tilde{J}(m) = \lambda + \alpha m$. The Firedrake implementation does this in three stages:

1. solve the forward Poisson problem to obtain the state u;
2. solve the adjoint problem to obtain λ;
3. compute the gradient as the function $\lambda + \alpha m$.

The numerical implementation mirrors the theoretical derivation. In particular, it exploits the fact that the adjoint equation lets us compute the derivative of the reduced functional without solving a separate tangent linear problem for every perturbation direction.

6.5 Taylor test for gradient verification

After implementing the adjoint and gradient in Firedrake, we carry out a Taylor test to verify that the computed gradient is correct. The Taylor test is a standard practical check for adjoint implementations [9].

Let δm be a perturbation direction and let $h > 0$ be a small parameter. The reduced functional satisfies the Taylor expansion

$$\tilde{J}(m + h\delta m) = \tilde{J}(m) + h\,\mathrm{d}\tilde{J}(m;\,\delta m) + \mathcal{O}(h^2). \tag{57}$$

If the gradient implementation is correct, the second-order remainder

$$R_2(h) := \left|\tilde{J}(m + h\delta m) - \tilde{J}(m) - h\,\mathrm{d}\tilde{J}(m;\,\delta m)\right| \tag{58}$$

decays as $\mathcal{O}(h^2)$ as $h \to 0$.

Using a smooth perturbation direction, we obtained the remainder sequence reported in Table 2.

h	second-order remainder	observed rate
$1.000 \cdot 10^{-1}$	$1.8923 \cdot 10^{-6}$	-
$5.000 \cdot 10^{-2}$	$4.7307 \cdot 10^{-7}$	2.0000
$2.500 \cdot 10^{-2}$	$1.1827 \cdot 10^{-7}$	2.0000
$1.250 \cdot 10^{-2}$	$2.9567 \cdot 10^{-8}$	2.0000
$6.250 \cdot 10^{-3}$	$7.3917 \cdot 10^{-9}$	2.0000

Table 2: Taylor test for the gradient of the reduced Poisson functional. The observed second-order decay of the Taylor remainder is strong evidence that the adjoint solve and the gradient computation are implemented correctly.

7 The Forward Problem for the Scalar Viscous Burgers Equation

Having done the full programme (forward solve, adjoint equation, reduced gradient, and numerical verification) through for the linear Poisson problem, we now repeat it for a harder, nonlinear model problem: the scalar viscous Burgers equation. Following the notation established earlier, we write the state variable as u and the control as m. This is a scalar simplification of the time-dependent viscous Burgers model considered by Schwedes et al. [13], whose treatment is vector-valued. The main new features compared with the Poisson problem are nonlinearity in u (through the convection term), time dependence (which we treat with backward Euler in Section 9), and a backward-in-time adjoint solve.

7.1 Strong formulation

Let $\Omega \subset \mathbb{R}$ be a bounded spatial domain and let $T > 0$. The scalar viscous Burgers equation reads

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = m \qquad \text{in } (0,T)\times\Omega, \tag{59}$$

subject to homogeneous Dirichlet boundary conditions

$$u = 0 \qquad \text{on } (0,T)\times\partial\Omega, \tag{60}$$

and initial condition

$$u(0,x) = u_0(x) \qquad \text{for } x \in \Omega. \tag{61}$$

Here, $\nu > 0$ is the viscosity parameter.

7.2 Weak formulation

For each fixed $t \in (0,T)$, we take the spatial state space to be $V := H_0^1(\Omega)$. Let $v \in V$ be a test function. Multiplying equation (59) by v and integrating over Ω gives

$$\int_\Omega \frac{\partial u}{\partial t}\, v\,\mathrm{d}x + \int_\Omega u\frac{\partial u}{\partial x}\, v\,\mathrm{d}x - \nu\int_\Omega \frac{\partial^2 u}{\partial x^2}\, v\,\mathrm{d}x = \int_\Omega m v\,\mathrm{d}x. \tag{62}$$

Integrating the diffusion term by parts and using $v = 0$ on $\partial\Omega$,

$$\int_\Omega \frac{\partial u}{\partial t}\, v\,\mathrm{d}x + \int_\Omega u\frac{\partial u}{\partial x}\, v\,\mathrm{d}x + \nu\int_\Omega \frac{\partial u}{\partial x}\frac{\partial v}{\partial x}\,\mathrm{d}x = \int_\Omega m v\,\mathrm{d}x \qquad \forall v \in V, \tag{63}$$

for almost every $t \in (0,T)$.

The convection term $u\,\partial_x u$ makes the Burgers dynamics qualitatively different from the linear Poisson problem. The local wave speed is the solution value u itself, so taller parts of the profile

travel faster than shorter ones. This carries the profile forward and compresses its trailing edge into a steep front, while the viscosity ν smooths that front rather than letting it become a true discontinuity, and slowly damps the amplitude. Figure 5 shows this behaviour for a representative forward solve. This nonlinear steepening, which the Poisson problem does not have, together with the backward-in-time adjoint it induces (Section 8), is the main new feature of the Burgers case.

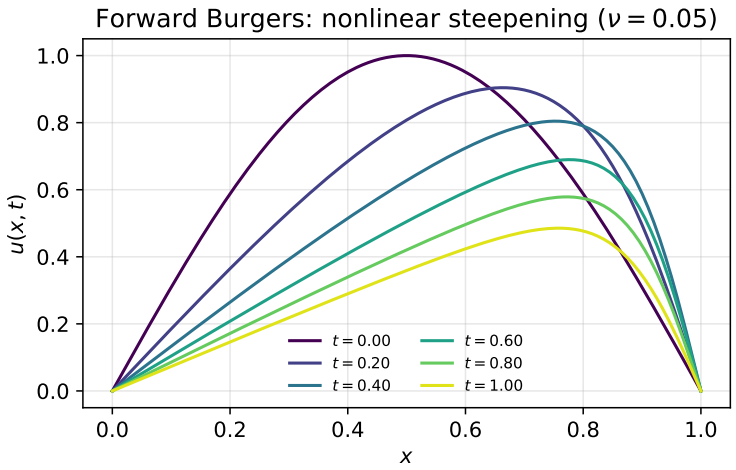

Figure 5: A representative solution of the forward Burgers equation with viscosity $\nu = 0.05$, starting from $u_0(x) = \sin(\pi x)$ with no forcing, shown at successive times.

8 Derivative of the Reduced Functional for the Burgers Problem

With the forward Burgers problem of Section 7 now in place, we extend the adjoint-based derivative analysis of Section 5 to the scalar Burgers problem. As before, the reduced functional is

$$\tilde{J}(m) := J(u(m), m), \tag{64}$$

now with the time-integrated tracking objective

$$J(u, m) := \tfrac{1}{2}\|u - d\|^2_{L^2((0,T)\times\Omega)} + \tfrac{\alpha}{2}\|m\|^2_{L^2((0,T)\times\Omega)}. \tag{65}$$

We take the control space to be

$$M := L^2((0, T) \times \Omega), \tag{66}$$

and the test space to be

$$\mathcal{V} := L^2(0, T; V). \tag{67}$$

The forward model operator F is the residual of equation (63), in direct analogy with Definition 5.1.

The derivation follows the same three steps as the Poisson case: we differentiate the constraint to obtain the tangent linear equation, introduce an adjoint variable chosen so that the unknown state variation cancels, and read the gradient off the result. The only new features are the nonlinearity of the convection term, which we now have to linearise, and the fact that the adjoint equation is solved backward in time.

8.1 Tangent linear equation

As in the Poisson case (Section 5.1), the first step is to relate the state variation δu to the control perturbation δm by differentiating the state equation. The one new feature is the convection term $u\,\partial_x u$: being nonlinear in u, its linearisation is where the real work of this step lies.

Let $\delta m \in M$ be a control perturbation and let $\delta u := \mathrm{d}u(m;\,\delta m)$ denote the corresponding variation in the state. Differentiating $F(u(m), m;\, v) = 0$ with respect to m gives us

$$\partial_u F(u, m;\, \delta u, v) + \partial_m F(u, m;\, \delta m, v) = 0 \qquad \forall v \in \mathcal{V}. \tag{68}$$

The derivative of the nonlinear convection term is

$$(u + \varepsilon\delta u)\frac{\partial}{\partial x}(u + \varepsilon\delta u) = u\frac{\partial u}{\partial x} + \varepsilon\left(\delta u\frac{\partial u}{\partial x} + u\frac{\partial \delta u}{\partial x}\right) + \mathcal{O}(\varepsilon^2), \tag{69}$$

so that

$$\partial_u F(u, m;\, \delta u, v) = \int_0^T \int_\Omega \left(\frac{\partial \delta u}{\partial t} v + \left(\delta u\frac{\partial u}{\partial x} + u\frac{\partial \delta u}{\partial x}\right) v + \nu\frac{\partial \delta u}{\partial x}\frac{\partial v}{\partial x}\right) \mathrm{d}x\,\mathrm{d}t, \tag{70}$$

while

$$\partial_m F(u, m;\, \delta m, v) = -\int_0^T \int_\Omega \delta m\, v \,\mathrm{d}x\,\mathrm{d}t. \tag{71}$$

Therefore, the tangent linear equation is

$$\int_0^T \int_\Omega \left(\frac{\partial \delta u}{\partial t} v + \left(\delta u\frac{\partial u}{\partial x} + u\frac{\partial \delta u}{\partial x}\right) v + \nu\frac{\partial \delta u}{\partial x}\frac{\partial v}{\partial x}\right) \mathrm{d}x\,\mathrm{d}t = \int_0^T \int_\Omega \delta m\, v \,\mathrm{d}x\,\mathrm{d}t \qquad \forall v \in \mathcal{V}, \tag{72}$$

together with the initial condition $\delta u(0, x) = 0$.

8.2 Adjoint equation

Exactly as in the Poisson case (Section 5.3), we introduce the adjoint to remove the state variation δu from the derivative. The new feature is temporal: integrating the time-derivative term by parts moves the derivative onto λ and forces a terminal condition, so the Burgers adjoint is solved backward in time.

The adjoint variable λ is defined by

$$\partial_u F(u, m;\, w, \lambda) = \partial_u J(u, m;\, w) \qquad \forall w \in \mathcal{V}, \tag{73}$$

where

$$\partial_u J(u, m;\, w) = \int_0^T \int_\Omega (u - d) w \,\mathrm{d}x\,\mathrm{d}t. \tag{74}$$

Substituting the expression for $\partial_u F$,

$$\int_0^T \int_\Omega \left(\frac{\partial w}{\partial t}\lambda + \left(w\frac{\partial u}{\partial x} + u\frac{\partial w}{\partial x}\right)\lambda + \nu\frac{\partial w}{\partial x}\frac{\partial \lambda}{\partial x}\right) \mathrm{d}x\,\mathrm{d}t = \int_0^T \int_\Omega (u - d) w \,\mathrm{d}x\,\mathrm{d}t \qquad \forall w \in \mathcal{V}. \tag{75}$$

We integrate the time-derivative term by parts in time,

$$\int_0^T \int_\Omega \frac{\partial w}{\partial t}\lambda \,\mathrm{d}x\,\mathrm{d}t = \int_\Omega w(T, x)\lambda(T, x)\,\mathrm{d}x - \int_\Omega w(0, x)\lambda(0, x)\,\mathrm{d}x - \int_0^T \int_\Omega w\frac{\partial \lambda}{\partial t}\,\mathrm{d}x\,\mathrm{d}t. \tag{76}$$

Admissible state variations satisfy $w(0, x) = 0$, so the initial term vanishes. We impose the terminal condition

$$\lambda(T, x) = 0. \tag{77}$$

Integrating the convection term by parts in space,

$$\int_\Omega \left(w\frac{\partial u}{\partial x} + u\frac{\partial w}{\partial x}\right)\lambda \,\mathrm{d}x = -\int_\Omega u\frac{\partial \lambda}{\partial x} w \,\mathrm{d}x, \tag{78}$$

where the boundary term vanishes because of the homogeneous Dirichlet condition. The weak adjoint equation therefore reads: find λ such that

$$\int_0^T \int_\Omega \left(-\frac{\partial \lambda}{\partial t} w - u \frac{\partial \lambda}{\partial x} w + \nu \frac{\partial \lambda}{\partial x} \frac{\partial w}{\partial x} \right) \mathrm{d}x \, \mathrm{d}t = \int_0^T \int_\Omega (u - d) w \, \mathrm{d}x \, \mathrm{d}t \qquad \forall w \in \mathcal{V}, \tag{79}$$

together with the terminal condition equation (77).

The corresponding strong form is

$$-\frac{\partial \lambda}{\partial t} - u \frac{\partial \lambda}{\partial x} - \nu \frac{\partial^2 \lambda}{\partial x^2} = u - d \qquad \text{in } (0,T) \times \Omega, \tag{80}$$

with $\lambda = 0$ on $(0,T) \times \partial\Omega$ and $\lambda(T,x) = 0$.

8.3 Derivative of the reduced functional

Applying the chain rule as in the Poisson case,

$$\mathrm{d}\tilde{J}(m;\, \delta m) = \partial_u J(u, m;\, \delta u) + \partial_m J(u, m;\, \delta m), \tag{81}$$

where

$$\partial_m J(u, m;\, \delta m) = \alpha \int_0^T \int_\Omega m \, \delta m \, \mathrm{d}x \, \mathrm{d}t. \tag{82}$$

Using the adjoint equation,

$$\partial_u J(u, m;\, \delta u) = \partial_u F(u, m;\, \delta u, \lambda), \tag{83}$$

and the tangent linear equation with $v = \lambda$,

$$\partial_u F(u, m;\, \delta u, \lambda) = -\partial_m F(u, m;\, \delta m, \lambda) = \int_0^T \int_\Omega \delta m \, \lambda \, \mathrm{d}x \, \mathrm{d}t. \tag{84}$$

So we have that

$$\mathrm{d}\tilde{J}(m;\, \delta m) = \int_0^T \int_\Omega (\lambda + \alpha m) \, \delta m \, \mathrm{d}x \, \mathrm{d}t \qquad \forall \delta m \in M. \tag{85}$$

8.4 The L^2-gradient

Since the control space is the Hilbert space $M = L^2((0,T) \times \Omega)$, the Riesz representation theorem identifies the reduced derivative equation (85) with a unique L^2-gradient. Comparing equation (85) with

$$\mathrm{d}\tilde{J}(m;\, \delta m) = (\nabla \tilde{J}(m),\, \delta m)_{L^2((0,T)\times\Omega)}, \tag{86}$$

we identify

$$\nabla \tilde{J}(m) = \lambda + \alpha m. \tag{87}$$

The reduced gradient has the same formal structure as in the Poisson case (see equation (51)), but the adjoint variable λ is now determined by the backward-in-time adjoint equation (equation (80)) which depends on the forward state u.

9 Numerical Implementation and Verification of the Burgers Problem

Having derived the weak formulation, the tangent linear and adjoint equations and the L^2-gradient of the reduced functional for the scalar viscous Burgers equation, we now implement the corresponding optimal control problem numerically. The aim is the same as in Section 6: verifying that the discretisation of the forward problem behaves as expected, and checking that the adjoint-based gradient computation is correct.

In contrast to what we did with the Poisson problem, where we provided both a plain Python implementation and a Firedrake implementation, we treat the Burgers problem at the lower level only. The main new difficulties specific to Burgers' equation are the nonlinearity, the time dependence and the backward-in-time adjoint solve. Implementing these ingredients directly in plain Python makes the structure of the method more transparent before moving to a higher-level framework.

9.1 Scalar Burgers numerically

We consider the one-dimensional spatial domain $\Omega = (0, 1)$ and a final time $T > 0$. The implementation discretises the scalar viscous Burgers equation in space using continuous piecewise linear finite elements on a uniform mesh, and in time using the backward Euler method.

Let $0 = t_0 < t_1 < \cdots < t_{N_t} = T$ denote the time grid, with uniform step

$$\Delta t = \frac{T}{N_t}. \tag{88}$$

At each implicit time level, the nonlinear problem is solved using Newton's method. If U^n denotes the finite element coefficient vector at time t_n, then the fully discrete forward problem at t_{n+1} has the form

$$M\frac{U^{n+1} - U^n}{\Delta t} + C(U^{n+1}) + \nu K U^{n+1} = F^{n+1}, \tag{89}$$

where M is the finite element mass matrix, K is the stiffness matrix, $C(U^{n+1})$ is the nonlinear convection contribution, and F^{n+1} is the load vector associated with the source term or control at time t_{n+1}.

As with the plain Python Poisson implementation, this code prioritises clarity over efficiency. Writing the assembly, the time stepping and the Newton linearisation explicitly makes the path from the nonlinear weak form to the algebraic system solved at each time step explicit. The implementation imposes homogeneous Dirichlet boundary conditions strongly at the two endpoints of the interval.

9.2 Manufactured solution and convergence study

To verify the forward implementation, we again use the method of manufactured solutions. We prescribe a smooth exact solution satisfying the homogeneous Dirichlet boundary conditions, and choose the corresponding source term so that this exact solution satisfies the Burgers equation.

We define

$$u_{\text{exact}}(t, x) = e^{-t}\sin(\pi x), \tag{90}$$

which vanishes at $x = 0$ and $x = 1$ and therefore satisfies the boundary conditions. Substituting into the strong form equation (59) gives the manufactured source term

$$m_{\text{exact}}(t, x) = -e^{-t}\sin(\pi x) + \pi e^{-2t}\sin(\pi x)\cos(\pi x) + \nu\pi^2 e^{-t}\sin(\pi x). \tag{91}$$

We solve the numerical problem on a sequence of uniformly refined meshes with $N \in \{10, 20, 40, 80\}$ elements. Since the problem is time-dependent, the temporal discretisation is refined at the same rate by taking $N_t = 8N$, so that Δt remains proportional to the mesh size h.

We record two error measures: the final-time $L^2(\Omega)$ error,

$$e_h^{\text{final}} := \|u_h(T) - u_{\text{exact}}(T)\|_{L^2(\Omega)}, \tag{92}$$

and a fully discrete space-time error,

$$e_h^{\text{st}} := \left(\Delta t \sum_{n=1}^{N_t} \|u_h^n - u_{\text{exact}}(t_n)\|_{L^2(\Omega)}^2 \right)^{1/2}. \tag{93}$$

The experimental order of convergence between two successive refinements is

$$q = \frac{\log(e_H/e_h)}{\log(h_H/h_h)}. \tag{94}$$

The observed results appear in Table 3.

N	N_t	h	e_h^{final}	rate	e_h^{st}	rate
10	80	0.1000	$5.840 \cdot 10^{-3}$	-	$1.838 \cdot 10^{-3}$	-
20	160	0.0500	$1.452 \cdot 10^{-3}$	2.0077	$4.590 \cdot 10^{-4}$	2.0027
40	320	0.0250	$3.577 \cdot 10^{-4}$	2.0197	$1.140 \cdot 10^{-4}$	2.0098
80	640	0.0125	$8.679 \cdot 10^{-5}$	2.0414	$2.810 \cdot 10^{-5}$	2.0209

Table 3: Manufactured-solution convergence test for the scalar Burgers forward solver. Both error measures decrease at an observed rate very close to 2, consistent with the expected behaviour of the fully discrete scheme for this smooth manufactured solution.

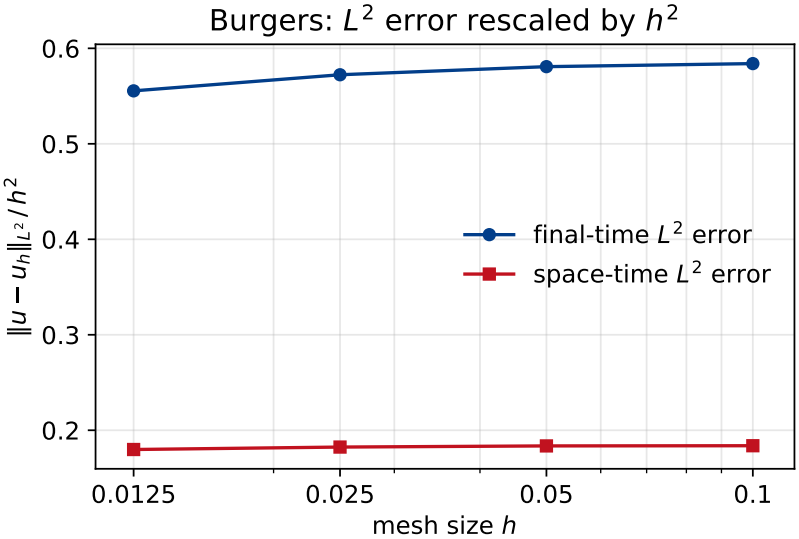

Figure 6: Manufactured-solution convergence of the forward Burgers solver (the data of Table 3), with each L^2 error rescaled by h^2. Both error measures plateau to a constant, confirming the $\mathcal{O}(h^2)$ rate; the reason the rate is second order despite the first-order backward Euler scheme is discussed below.

Although the backward Euler discretisation is only first order in time, the observed second-order rate is governed by the P_1 spatial discretisation. Taking $N_t = 8N$ makes the time step $\Delta t = Th/8$ small relative to the mesh size, so the $\mathcal{O}(\Delta t)$ temporal error remains well below the $\mathcal{O}(h^2)$ spatial error over the resolutions tested. Refining the time step alone, on a fixed fine mesh, recovers the expected first-order temporal rate, and refining the mesh alone, on a fixed fine time grid, the second-order spatial rate.

9.3 Discrete adjoint solve and gradient computation

We implement the discrete adjoint equation and compute the gradient of the reduced functional at the fully discrete level, consistent with the backward Euler time discretisation of the forward problem.

If D^n denotes the discrete desired state at time t_n and m^n the discrete control, the fully discrete tracking functional is

$$J_h(U, m) = \frac{\Delta t}{2} \sum_{n=1}^{N_t} \|U^n - D^n\|_M^2 + \frac{\alpha \Delta t}{2} \sum_{n=1}^{N_t} \|m^n\|_M^2, \tag{95}$$

where $\|v\|_M^2 := v^T M v$ is the mass-matrix representation of the $L^2(\Omega)$ norm.

We generate the desired state by first choosing a smooth reference control, solving the forward Burgers problem for that control, and taking the resulting discrete state trajectory as the target trajectory. We then choose a different trial control and solve the forward problem again in order to compute the state, the objective value, the adjoint and the gradient.

As expected for a time-dependent problem, the discrete adjoint variables are computed by solving a sequence of linear systems backward in time, starting from a zero terminal condition. The forward Burgers problem is nonlinear and is solved forward in time, while the adjoint problem is linearised about the computed state and is solved backward in time [13, Sect. 1.4.5].

The resulting discrete gradient has the form

$$g^n = \alpha m^n + \frac{1}{\Delta t} p^n, \tag{96}$$

where p^n is the discrete adjoint variable at time level t_n. This is the fully discrete version of the continuous expression $\nabla \tilde{J}(m) = \lambda + \alpha m$ derived earlier.

For the representative trial control used in the experiments, the computed objective value was

$$J_h(m) = 1.323574 \times 10^{-4}, \tag{97}$$

and the average number of Newton iterations required in the forward solve was approximately 3 per time step, indicating that the nonlinear solver converged reliably for the chosen problem parameters.

9.4 Taylor test for gradient verification

After implementing the discrete adjoint and gradient, we carry out a Taylor test in the same spirit as in Section 6.5.

Let δm be a perturbation direction in the discrete control space and let $h > 0$. The reduced functional satisfies

$$\tilde{J}_h(m + h\delta m) = \tilde{J}_h(m) + h\, \mathrm{d}\tilde{J}_h(m;\, \delta m) + \mathcal{O}(h^2), \tag{98}$$

and if the gradient implementation is correct, the second-order remainder

$$R_2(h) := \left| \tilde{J}_h(m + h\delta m) - \tilde{J}_h(m) - h\, \mathrm{d}\tilde{J}_h(m;\, \delta m) \right| \tag{99}$$

decays as $\mathcal{O}(h^2)$ as $h \to 0$.

Using a smooth perturbation direction normalised in the discrete control inner product, we obtained the results of Table 4.

h	second-order remainder	observed rate
$1.000 \cdot 10^{-1}$	$6.7317 \cdot 10^{-5}$	-
$5.000 \cdot 10^{-2}$	$1.6828 \cdot 10^{-5}$	2.0001
$2.500 \cdot 10^{-2}$	$4.2068 \cdot 10^{-6}$	2.0001
$1.250 \cdot 10^{-2}$	$1.0517 \cdot 10^{-6}$	2.0000
$6.250 \cdot 10^{-3}$	$2.6291 \cdot 10^{-7}$	2.0000

Table 4: Taylor test for the discrete reduced functional in the scalar Burgers problem. The observed second-order decay of the Taylor remainder is strong evidence that the discrete adjoint solve and gradient computation are implemented correctly.

Together, the verifications of Sections 6 and 9 show that the adjoint-based optimisation framework developed in Sections 5 and 8 extends successfully from a linear, stationary problem to a nonlinear, time-dependent one, from the simple Poisson example to the broader questions of PDE-constrained optimisation and mesh dependence taken up next in Section 10.

10 Mesh Dependence in PDE-Constrained Optimisation

So far, we have derived, discretised and verified the adjoint-based gradient for both the Poisson and the scalar viscous Burgers optimal control problems. In each case, the L^2-gradient $\nabla \tilde{J}(m) = \lambda + \alpha m$ was identified using the Riesz representation theorem (Theorem 2.7). That identification is not unique: it depends on the choice of Hilbert space structure placed on the control space M. In this section we examine the practical consequences of that choice when the reduced functional is minimised numerically. Following Schwedes et al. [13, Chap. 2], we show that two different but mathematically valid choices of inner product give optimisation algorithms whose iteration counts behave very differently as the mesh is refined.

10.1 The two Riesz maps

Sections 2.4 and 5.5 established that the Fréchet derivative $\mathrm{d}\tilde{J}(m)$ is an element of the dual space M^*, and that the Riesz map $\mathcal{R}_H : M^* \to M$ identifies each derivative with a primal element of M. When $M = L^2(\Omega)$ is treated as an L^2 Hilbert space, the Riesz map $\mathcal{R}_{L^2}$ corresponds in the discrete setting to multiplication by the inverse of the Galerkin mass matrix [13, Eq. (1.125)]. When the same coefficient vector is regarded instead as an element of $\mathbb{R}^d$ under the standard Euclidean (ℓ^2) inner product, no mass matrix appears and the Riesz map is the identity.

The two choices give us primal gradients that differ at every mesh refinement: the ℓ^2 representer carries the local cell sizes encoded in the diagonal of the mass matrix, while the L^2 representer does not. Search directions and stopping criteria based on gradient norms therefore behave differently in the two settings. Schwedes et al. [13, Sect. 2.1] note that "many well-established continuous optimisation packages are not designed for function-based optimisation and are hard-coded to apply the Euclidean (ℓ^2) inner product", and explicitly list `scipy.optimize`, IPOPT and TAO among them. The Moola library [3] and recent versions of pyadjoint allow the user to specify the Hilbert space inner product on the control space.

10.2 Hypothesis

We test the prediction of Schwedes et al. [13, Sect. 2.2]: for a fixed gradient-norm convergence threshold, the iteration count of a quasi-Newton method depends on which Riesz map is used to interpret the derivative as a primal-space gradient. Concretely:

(I) When the ℓ^2 Riesz map is used, the iteration count grows with the mesh ratio $h_{\max}/h_{\min}$ (mesh-dependent convergence).

(II) When the L^2 Riesz map is used, the iteration count is bounded uniformly in $h_{\max}/h_{\min}$ (mesh-independent convergence).

10.3 The benchmark: Table 2.2 of Schwedes et al. [13]

The experiments of this chapter are a reproduction of Table 5, which is Table 2.2 of Schwedes et al. [13]. We reproduce it here so that it is clear exactly what we are comparing against.

inner product	implementation	$h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	Scipy (BFGS)	27	47	75	97	133	189
ℓ^2	TAO (LMVM)	42	60	77	111	131	155
ℓ^2	IPOPT (Interior Point)	28	38	65	88	117	139
L^2	Moola (BFGS)	22	20	22	23	23	27

(a) $H = L^2(\Omega)$

inner product	implementation	$h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	Scipy (BFGS)	25	53	118	222	244	594
ℓ^2	TAO (LMVM)	38	38	110	144	141	384
ℓ^2	IPOPT (Interior Point)	31	38	85	127	182	443
H^1	Moola (BFGS)	4	4	4	4	4	4

(b) $H = H^1(\Omega)$

Table 5: Reproduced from Table 2.2 of Schwedes et al. [13]: iteration counts to reach $\|\mathcal{R}_H(J')\|_H \leq 10^{-7}$ for four optimisation packages on non-uniform meshes of increasing grading ratio $h_{\max}/h_{\min}$, under two choices of control-space inner product. The history length for the Hessian approximation is ten, except for TAO/LMVM where it is five. The three ℓ^2 rows grow with the mesh ratio, whereas the Moola row, which uses the function-space inner product, stays flat.

The columns sweep the grading ratio $h_{\max}/h_{\min}$ from 4 to 128, a larger value meaning a more strongly non-uniform mesh. Each row is one optimisation package together with the inner product it uses internally. The first three rows, Scipy, TAO and IPOPT, all use the Euclidean ℓ^2 inner product of the coefficient vector, which is the default in general-purpose optimisation libraries. The last row, Moola, uses the function-space inner product, the L^2 norm in panel (a) and the H^1 norm in panel (b), through its Riesz map. The threshold $\varepsilon = 10^{-7}$ is applied to the gradient measured in that same norm, $\|\mathcal{R}_H(J')\|_H$, so that every row is stopped on a comparable quantity. The pattern is the point of the whole chapter: the three ℓ^2 rows grow steadily as the mesh becomes more graded, while the Moola row stays essentially constant, around twenty in panel (a) and exactly four in panel (b).

We do not reproduce every cell, and it is worth saying why. The scientific content is the contrast between the ℓ^2 rows and the function-space row, not the four packages individually. Scipy, TAO and IPOPT are three implementations of the same ℓ^2 choice, so one representative is enough to show the growth, and we take Scipy's L-BFGS-B for that role (Tables 7 and 8). For the function-space row we cannot use Moola directly: it hard-codes a `dolfin` dependency and does not install in a Firedrake environment, so we reimplement its function-space BFGS ourselves, as the Hilbert-space L-BFGS of Section 10.6, which takes every inner product in the chosen norm and makes explicit where the metric enters. TAO/LMVM is not discarded but promoted to a study of its own (Section 10.9 and Section 11), since whether TAO can be made mesh-independent, and how, is one of the contributions of this work. Finally, the table is a snapshot of the 2017 software. The packages have since moved on, and our experiments use PETSc `v3.24`, so we expect to reproduce the qualitative growth-versus-flat contrast rather than to match the counts cell for cell. In particular the TAO/LMVM row was run at history length five, which Section 10.9 identifies as the very reason it appears mesh-dependent.

10.4 Mesh generation

We use two families of non-uniform mesh, and the order in which we take them is deliberate. The deterministic graded meshes come first because they are the meshes behind Schwedes' Table 2.2: their refinement is fixed and their $h_{\max}/h_{\min}$ ratio can be set to a prescribed value, which is what a faithful reproduction of the headline result requires. The randomly refined meshes come second, as a robustness check. They discard the smooth, structured grading of the first family and refine cells irregularly, so that any mesh-dependence we observe cannot be an artefact of one tidy grading pattern; if the same behaviour survives on irregular meshes, it is a property of the inner product rather than of the mesh construction. We describe the two families in turn.

The deterministic graded meshes are built on a structured base. The unit square is meshed by Firedrake's `UnitSquareMesh` into an 8×8 grid of squares, each split into two triangles along a diagonal. A fixed interior sub-square $[0.4, 0.6]^2$ is then refined by repeated longest-edge (Rivara) bisection [12]: in each round, every triangle whose centroid lies in the sub-square is bisected, and the bisection is propagated to neighbours so that the mesh stays conforming, with no hanging nodes. After $2\log_2 r$ rounds the cells inside the sub-square are a factor r finer than those outside, so the realised ratio $h_{\max}/h_{\min}$ is *exactly* the prescribed power of two rather than an approximation to it. Three of the six meshes are shown in Figure 7.

This construction uses no external mesher and needs no post-processing: the six meshes for $r \in \{4, 8, 16, 32, 64, 128\}$ are generated directly and written as Gmsh 2.2 `.msh` files [4], which Firedrake loads without modification. The cell counts and realised ratios are reported in Table 6.

target r	# cells	realised $h_{\max}/h_{\min}$
4	256	4.00
8	640	8.00
16	1 712	16.00
32	5 776	32.00
64	22 608	64.00
128	87 312	128.00

Table 6: The six deterministic graded meshes used in the experiments. The realised $h_{\max}/h_{\min}$ is an exact power of two by construction, and is measured from the cell-diameter field via `CellDiameter` in Firedrake.

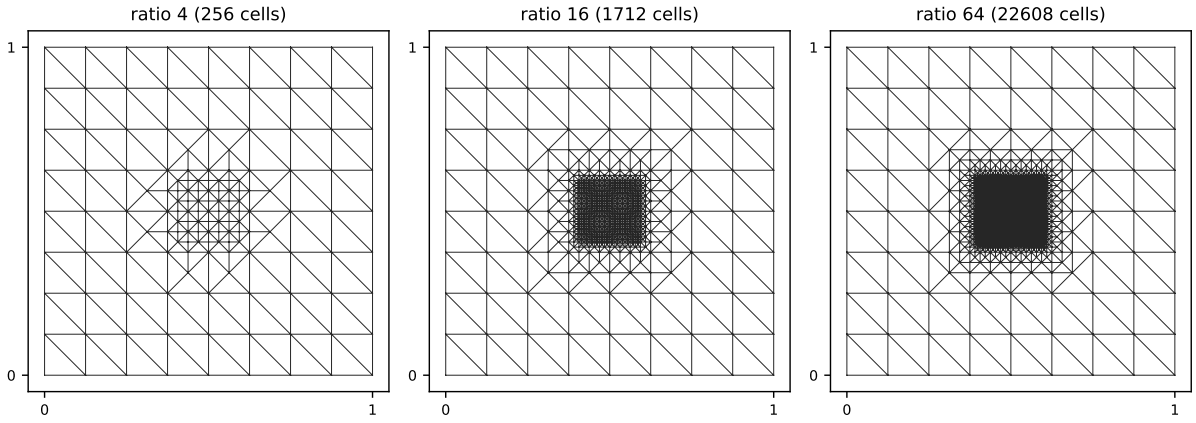

Figure 7: Three of the six deterministic graded meshes of Table 6, at ratios $r = 4, 16, 64$. The structured 8×8 base, squares split into triangles, is refined inside the central sub-square $[0.4, 0.6]^2$ by longest-edge bisection, so the smallest cells are a factor r finer than the outer ones. At $r = 64$ the refined region is so fine it appears almost solid.

The deterministic graded meshes above correspond to the boundary-refinement scheme of Schwedes et al. [13, Sect. 2.3.1], which is the scheme behind the Table 2.2 experiment reproduced in this section. We also implemented the random refinement scheme of Schwedes et al. [13, Sect. 2.3.2.1], in which each cell is independently marked with probability p and refined by longest-edge (Rivara) bisection [12], the refinement propagating to neighbours along the longest-edge propagation path so that the mesh stays conforming. Applied over the whole domain, this scheme cannot reach the high ratios of the graded meshes: the realised $h_{\max}/h_{\min}$ saturates near 5.7 even as the cell count grows, and lowering p only lifts the ceiling to roughly 8–11. The reason is the propagation: refining one cell eventually forces most of the domain to refine, so $h_{\max}$ shrinks almost as fast as $h_{\min}$. Restricting the marking to a fixed interior sub-square removes this ceiling and reaches the same range of ratios as the graded meshes, and the resulting interior random meshes are used for the corroborating experiments of Section 12.5; three of them are shown in Figure 8.

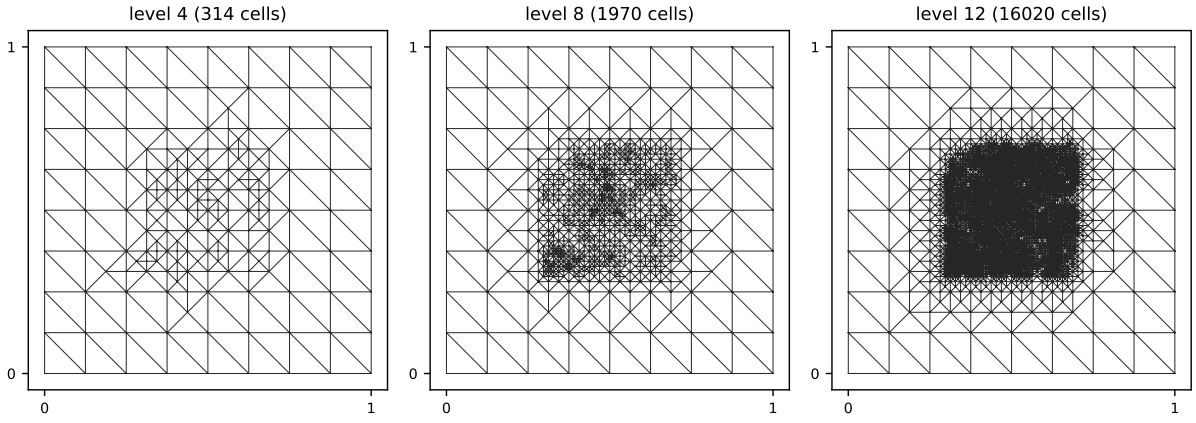

Figure 8: Three of the interior random-refined meshes used for the matching experiments of Section 12.5, at refinement levels 4, 8 and 12 (realised $h_{\max}/h_{\min} = 4, 16, 64$). Cells inside the sub-square $(0.3, 0.7)^2$ are marked at random and refined by longest-edge bisection, giving the irregular pattern seen here, in contrast to the smoothly graded meshes of Figure 7. The refinement is kept away from the domain boundary, so it never interacts with the Dirichlet conditions.

10.5 Implementation

Testing the hypothesis in code requires pinning down two things: which inner product each algorithm uses internally, and a stopping criterion applied identically to every algorithm regardless of that choice. We describe how both are handled, together with the practical obstacles that arise in routing the problem through TAO/LMVM.

The experiments are built on Firedrake [11] and its algorithmic-differentiation extension pyadjoint [9]. The choice of inner product on the control is made through pyadjoint's `Control`, whose `riesz_map` argument, one of `"l2"`, `"L2"` or `"H1"`, selects which discrete Riesz map pyadjoint applies when it converts the dual derivative $\mathrm{d}\tilde{J}(m)$ into a primal gradient:

```
Jhat = ReducedFunctional(
    J, Control(m, riesz_map=riesz_map), tape=tape,
)
```

The forward Poisson problem is solved by a direct LU factorisation, so the discrete gradient is computed to machine precision; the convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_H(J')\|_H$ is therefore never obscured by Krylov-solver noise.

Following Schwedes et al. [13, pp. 67–68], we apply the convergence criterion panel-wide rather than per algorithm: every algorithm in a panel is stopped on the same gradient norm $\|\mathcal{R}_H(J')\|_H \leq 10^{-7}$, regardless of the inner product it uses internally. For an algorithm hard-coded to the ℓ^2 inner product, such as SciPy's L-BFGS-B, the derivative is converted to the appropriate inner product *outside* the algorithm. We reproduce this by setting SciPy's native tolerances to 10^{-30} so that they never trigger, while a callback re-evaluates $\mathcal{R}_H(J')$ at the current iterate, computes its H-norm, and raises a sentinel exception once the threshold is met.

The problem can also be routed through PETSc's TAO/LMVM solver, although this requires two non-default settings. The natural recipe combines `Control(m, riesz_map="L2")` with pyadjoint's TAO wrapper. Pyadjoint then installs the discrete Riesz map M_h^{-1} in two places: as the gradient-norm metric in the convergence test, through `tao.setGradientNorm`, and as the initial Hessian B_0 in the L-BFGS recursion, through `setLMVMH0`. The LMVM matrix maintains its secant pairs in the inner product induced by B_0, so it performs, in principle, function-space L-BFGS. Two issues nonetheless arise in practice.

The first is mechanical. The option `tao_lmvm_mat_lmvm_hist_size`, which controls the number of secant pairs retained, is silently ignored when it is passed through pyadjoint's `parameters` dictionary, owing to an options-prefix mismatch between pyadjoint and the internal PETSc matrix. It must instead be registered on the global PETSc options database before the `TAOSolver` is constructed:

```
PETSc.Options().setValue('-tao_lmvm_mat_lmvm_hist_size', 100)
```

The second is numerical. The default history of five retains only five secant pairs, so the rank-five quasi-Newton update captures only the five largest eigenvalues of the discrete Hessian. On the graded meshes of Table 6 the Hessian has a long-tailed spectrum, and a rank-five approximation gives a poorly conditioned search direction. The consequence is slow convergence rather than outright failure: with enough outer iterations the gradient norm does reach machine precision, as the probe described in Section 10.9 confirms. At the threshold $\varepsilon = 10^{-7}$, however, the iteration count grows from around 130 on a Cartesian mesh to several hundred on the most strongly graded meshes. Enlarging the history to 100 restores a mesh-independent count of roughly 30 across the range.

TAO/LMVM with these corrections reproduces Schwedes' pattern, but its rank-two update lives inside a PETSc C kernel and is hard to inspect from Python. To make the role of the inner

product completely explicit, we also implement BFGS in a Hilbert space directly, following Schwedes et al. [13, Sect. 1.5.4.1]. Every dot product in the two-loop recursion is taken in the chosen inner product $(\cdot,\,\cdot)_H$ rather than in ℓ^2, which is the distinguishing feature of Schwedes' Moola row. The algorithm is stated in Section 10.6.

10.6 BFGS in a Hilbert space

This is the reference implementation: a limited-memory BFGS in which the metric is completely explicit, so that one can read off at every step where the inner product enters.

We take H to be either $L^2(\Omega)$ or $H^1(\Omega)$, with the inner products

$$(a,\,b)_{L^2} = \int_\Omega a\,b\,\mathrm{d}x, \qquad (a,\,b)_{H^1} = \int_\Omega \big(a\,b + \nabla a\cdot\nabla b\big)\,\mathrm{d}x. \tag{100}$$

At the current iterate $m_k \in M$, let $g_k := \mathcal{R}_H(\mathrm{d}\tilde{J}(m_k))$ denote the primal Riesz representer of the derivative. The algorithm stores the most recent $\ell \le 5$ curvature pairs

$$s_j := m_{j+1} - m_j \in M, \qquad y_j := g_{j+1} - g_j \in M, \qquad \rho_j := \frac{1}{(s_j,\,y_j)_H}, \tag{101}$$

and forms the search direction p_k by the two-loop recursion

$$q \leftarrow g_k, \tag{102}$$

for $j = k-1, k-2, \dots, k-\ell$:

$$\alpha_j \leftarrow \rho_j\,(s_j,\,q)_H, \tag{103}$$
$$q \leftarrow q - \alpha_j\,y_j, \tag{104}$$

then

$$z \leftarrow \gamma_k\,q, \tag{105}$$

for $j = k-\ell, \dots, k-1$:

$$\beta_j \leftarrow \rho_j\,(y_j,\,z)_H, \tag{106}$$
$$z \leftarrow z + (\alpha_j - \beta_j)\,s_j, \tag{107}$$

and finally

$$p_k \leftarrow -z, \tag{108}$$

in which every inner product is taken in $(\cdot,\,\cdot)_H$.

The initial Hessian is rescaled at each iteration by

$$\gamma_k = \frac{(s_{k-1},\,y_{k-1})_H}{(y_{k-1},\,y_{k-1})_H}, \tag{109}$$

the standard L-BFGS scaling of Schwedes et al. [13, Sect. 1.5.4.2], which adapts B_0 to the local curvature. Omitting it causes the algorithm to plateau several orders of magnitude above the threshold, even on a well-conditioned mesh.

The step length is chosen by a backtracking Armijo line search: starting from $\alpha = 1$, the trial step is halved until the sufficient-decrease condition

$$\tilde{J}(m_k + \alpha\,p_k) \le \tilde{J}(m_k) + c_1\,\alpha\,(g_k,\,p_k)_H, \qquad c_1 = 10^{-4}, \tag{110}$$

holds. Curvature pairs with $(s_j, y_j)_H \leq 0$ are discarded, and the iteration terminates once $\|g_k\|_H \leq \varepsilon = 10^{-7}$.

The Riesz representer g_k is obtained from the dual derivative by a direct LU factorisation of the inner-product matrix, M_h for L^2 and $M_h + K_h$ for H^1. Firedrake's default `RieszMap` instead uses a Krylov solve, whose tolerance would leak into the convergence test; the direct factorisation removes that tolerance, so any failure to reach $\varepsilon = 10^{-7}$ is attributable to the optimiser rather than to the Riesz solve.

10.7 Numerical experiments: Panel (a), $H = L^2(\Omega)$

We reproduce Panel (a) of Table 2.2 in Schwedes et al. [13], in which the control space is treated as the L^2 Hilbert space. The state equation is the weak Poisson problem equation (21), and the objective is equation (23) with target $d(x, y) = \sin(\pi x)\sin(\pi y)$ and Tikhonov parameter $\alpha = 10^{-4}$. The control is initialised at $m \equiv 0$, and every run is stopped at the threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$, with an L-BFGS history of five. Two algorithms are compared: SciPy's L-BFGS-B [2, 14], which operates on the ℓ^2 coefficient vector, with the convergence test performed externally in L^2; and the Hilbert-space L-BFGS of Section 10.6 with $H = L^2(\Omega)$. The results are collected in Table 7 and plotted in Figure 10, and the recovered solution of the problem is shown in Figure 9.

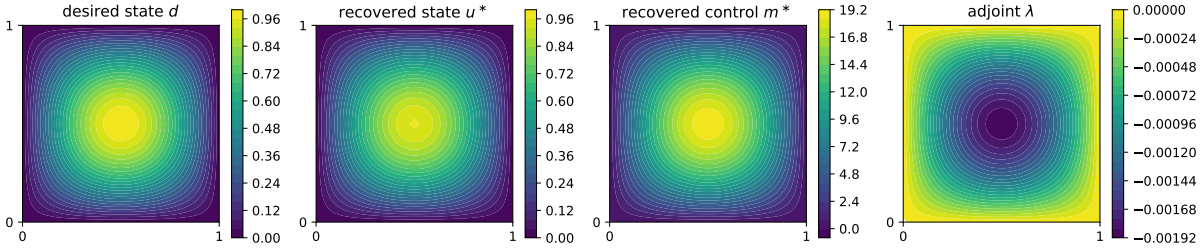

Figure 9: The Poisson optimal control problem of Panel (a), solved to optimality on a uniform 48×48 mesh. The recovered control m^* drives the state u^* to match the desired state d; the adjoint λ is small because the state already tracks the target closely.

inner product	implementation	target $h_{\max}/h_{\min}$				
		4	8	16	32	64
ℓ^2	SciPy (L-BFGS-B, external check)	58	135	282	518	1040
L^2	Hilbert-space L-BFGS	6	6	6	6	6

Table 7: Iteration counts at convergence for the Poisson optimal control problem on the deterministic graded meshes of Table 6. Both rows used the same problem data, the same forward LU solver, the same convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$ and the same L-BFGS history of five. The only difference between the rows is the inner product used internally by the algorithm.

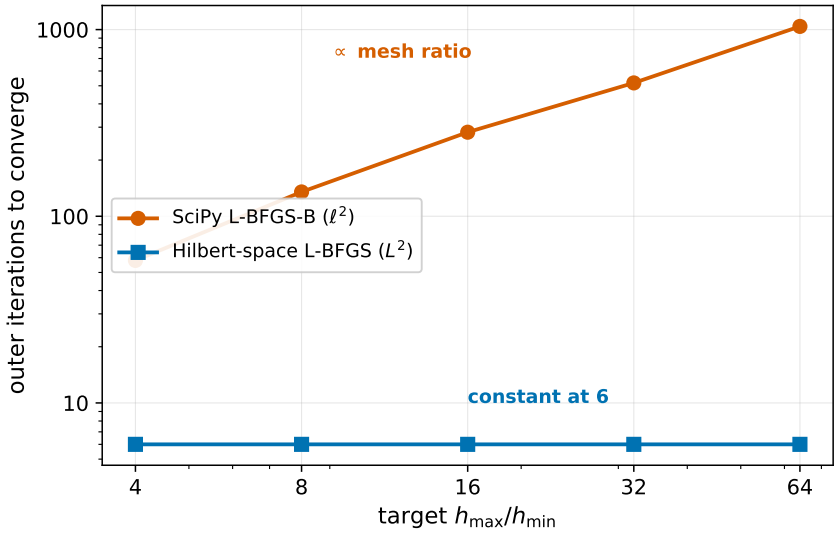

Figure 10: Iteration counts of Table 7 against the mesh ratio. On the ℓ^2 inner product the count doubles at every refinement; on the L^2 inner product it stays flat between seven and eight. The only difference between the two runs is the inner product used inside the optimiser.

The ℓ^2 row roughly doubles at every refinement (by factors of 2.33, 2.09, 1.84 and 2.01), matching the polynomial-in-$h_{\max}/h_{\min}$ growth derived in Schwedes et al. [13, Sect. 2.2.4], while the L^2 row is constant at six iterations across the whole sweep. This is the contrast predicted by hypotheses (I) and (II) of Section 10.2. For comparison, Schwedes et al. [13, Table 2.2(a)] report $27, 47, 75, 97, 133$ for their SciPy row and $22, 20, 22, 23, 23$ for their Moola row at the same target ratios. The qualitative pattern reproduces; the absolute counts differ because of implementation details, chiefly the line search: our backtracking Armijo search, which enforces only sufficient decrease, converges in fewer outer iterations than Moola's strong-Wolfe line search [3], the Moré–Thuente `dcsrch` routine [10] with sufficient-decrease constant `ftol` $= 10^{-3}$ and curvature constant `gtol` $= 0.9$, which additionally enforces a curvature condition.

10.8 Numerical experiments: Panel (b), $H = H^1(\Omega)$

Panel (b) of Table 2.2 in Schwedes et al. [13] repeats the experiment with the H^1 inner product on the control space. A detail that is easily overlooked is that the objective functional of Schwedes et al. [13, Eq. (1.219)] regularises in the *same* norm as the metric: it reads $\frac{1}{2}\|u-d\|_{L^2}^2 + \frac{\alpha}{2}\|m\|_M^2$ with $M = L^2$ or H^1. Moving from Panel (a) to Panel (b) therefore changes the regularisation term from $\frac{\alpha}{2}\int_\Omega m^2$ to

$$\tfrac{\alpha}{2}\|m\|_{H^1}^2 = \tfrac{\alpha}{2}\int_\Omega \left(m^2 + |\nabla m|^2\right) \mathrm{d}x. \tag{111}$$

We pose this H^1-regularised problem and, as in Panel (a), compare two algorithms stopped on the same threshold $\varepsilon = 10^{-7}$, now measured in the H^1 norm $\|\mathcal{R}_{H^1}(J')\|_{H^1}$: SciPy's ℓ^2 L-BFGS-B, with the convergence test performed externally in H^1, and the Hilbert-space L-BFGS of Section 10.6 with $H = H^1(\Omega)$. The results are given in Table 8.

inner product	implementation	target $h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	SciPy (L-BFGS-B, external check)	44	59	107	188	451	1075
H^1	Hilbert-space L-BFGS	4	4	4	4	4	4

Table 8: Iteration counts at convergence for the H^1-regularised Poisson optimal control problem on the deterministic graded meshes of Table 6, at the threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{H^1}(J')\|_{H^1}$. Both rows solve the same H^1-regularised problem and reach the same discrete optimum; they differ only in the inner product used internally. The Hilbert-space L-BFGS row is mesh-independent at four iterations, reproducing the constant Moola row of Schwedes et al. [13, Table 2.2(b)] exactly.

The Hilbert-space L-BFGS row is flat at four iterations across the entire sweep, over which the realised mesh ratio grows by a factor of 32 and the number of cells by a factor of about 340. This reproduces the mesh-independent behaviour of Schwedes' Moola row exactly: their Table 2.2(b) also reports a constant four. The ℓ^2 row, by contrast, grows by roughly a factor of two at each refinement, in the same manner as the ℓ^2 row of Panel (a); the two algorithms reach the same discrete optimum and differ only in the inner product used internally.

It is essential that the regularisation and the metric are taken in the same norm. Applying the H^1 metric to the L^2-regularised problem of Panel (a), that is, changing the inner product without changing the objective, is a mismatch: the Hilbert-space L-BFGS still reaches the same L^2-regularised optimum, but takes $100, 102, 103, 136, 113$ and 141 iterations across the six meshes, more than an order of magnitude above the four to six of the matched problem. This mismatch, rather than any deficiency of the H^1 Riesz map itself, accounts for the large iteration counts recorded in an earlier version of this chapter. When the metric matches the norm in which the problem is posed, the H^1 Riesz map delivers mesh-independent convergence, exactly as the L^2 Riesz map does in Panel (a).

10.9 TAO/LMVM revisited: diagnosing the original stagnation

Section 10.5 noted that TAO/LMVM with `Control(m, riesz_map="L2")` fails to reproduce the Moola row unless the LMVM history is enlarged. This subsection records the diagnostic chain that led to that conclusion, which was misdiagnosed once before it was reached.

The first hypothesis was that LMVM stagnates at a gradient-norm floor below which round-off in the adjoint tape prevents further reduction. To test it, TAO/LMVM was replaced by TAO/NLS (an exact Newton method with CG-based Hessian-vector products) and the tolerance was tightened to $\varepsilon = 10^{-30}$. Because the Poisson problem is quadratic, a single Newton step lands at the discrete optimum, up to the Newton solver tolerance. Evaluating the gradient at that iterate gave $\|\mathcal{R}_{L^2}(J')\|_{L^2} \approx 10^{-18}$ on every mesh in the sweep, which places the adjoint-tape noise floor at machine precision rather than at the 10^{-5} where LMVM had stagnated. The stagnation is therefore not a noise floor.

The second hypothesis was that the rank-five quasi-Newton update is too coarse for the graded Hessian spectrum. The original test of this hypothesis (a synthetic exponential stretch with a history sweep $h \in \{5, 25, 50, 100, 200\}$ and tolerance 10^{-30}) proved invalid: the synthetic stretch collapses cells onto the Dirichlet corner, where both the state u and the target d vanish, so the over-resolved boundary region carries no objective information and the apparent stagnation is in part a degeneracy. Repeating the history sweep on the deterministic graded meshes of Table 6 remains to be done; the expected behaviour is that $h = 5$ stagnates above the discrete optimum on the strongly graded meshes while $h \geq 25$ reaches it on all of them. The survey of TAO solvers

on the deterministic meshes (Table 9) supports this, and separates two distinct ingredients. The `lmvm` solver carries the mass-matrix seed installed by `Control(m, riesz_map="L2")` in every run, so its growth to several hundred iterations at the default history of five and its mesh-independence at history 100 ($17, 17, 17$ and 17 iterations) differ only in the history; this isolates the limited-memory truncation as the cause of the LMVM failure. The related `bqnls` solver is useful for the opposite reason: pyadjoint installs the mass-matrix seed only for the `lmvm` and `blmvm` solvers, so `bqnls` runs without it. At the default history it is mesh-dependent ($30, 76, 130, 163$); enlarging the history to 100 does not remove the growth ($23, 49, 80$ at $r = 4, 16, 64$, still rising); and installing the seed $J_0 = M_h$ by hand, through `Mat.setLMVMJ0` on the LMVM matrix returned by `getLMVMMat`, flattens it to a constant six across the sweep. The mesh-independence of TAO/LMVM therefore rests on two ingredients that these solvers isolate separately: a sufficiently rich limited-memory history, and the mass-matrix seed that carries the metric.

The history size is itself awkward to set. Passing `tao_lmvm_mat_lmvm_hist_size` through pyadjoint's `parameters` dictionary does not propagate to the LMVM sub-matrix, because of an options-prefix mismatch between pyadjoint and the internal PETSc matrix object attached to the LMVM `Tao`. The option must instead be registered on the global PETSc options database before the `TAOSolver` is constructed,

```
PETSc.Options().setValue('-tao_lmvm_mat_lmvm_hist_size', 100)
```

or, alternatively, the pyadjoint TAO solver may be constructed with an empty options prefix. We use the former, as in Section 10.5.

The Poisson problem is quadratic in m, and quasi-Newton methods are unusually effective on quadratics regardless of the inner product used, so the finding should be checked on a non-quadratic problem. The natural candidate is the one-dimensional viscous Burgers control problem. The earlier Burgers sweep, however, used a synthetic exponentially-stretched interval, whose cells collapse onto the Dirichlet endpoint where both the initial data u_0 and the target d vanish (the same boundary-collapse pathology identified in two dimensions), and its results are not included here. A direct equivalent is not available, since the graded-mesh construction here is two-dimensional. A future revision could instead cluster cells toward an interior point of the interval, so that no Dirichlet collapse occurs; move to a two-dimensional Burgers control problem discretised on the same deterministic graded meshes as the Poisson problem; or restrict the claims to Poisson control. On the Poisson evidence of Table 9, the refined conclusion is that the mesh-dependence of Schwedes' Table 2.2(a) is really about solver classes that fail to capture enough Hessian information on graded meshes for diffusion-dominated problems, rather than a universal ℓ^2 pathology of every iterative solver.

To probe this, the same Poisson problem and grading sweep were run through every PETSc TAO solver that accepts an unconstrained, or bound-constrained, problem of this class; the results are summarised in Table 9. The Newton-class solvers, and `lmvm` with the history fix, are mesh-independent, whereas the nonlinear conjugate gradient solver `cg` shows the same mesh-dependent growth as LMVM at the default history. The pathology is therefore not specific to LMVM.

type	TAO solver	$h_{\max}/h_{\min}$			
		4	16	64	128
Newton line search	`nls`	2	2	2	2
bounded Newton line search	`bnls`	1	1	1	1
bounded Newton trust region	`bntr`	8	8	9	9
bounded Newton trust + line	`bntl`	8	8	9	9
bounded Newton Krylov	`bnk`	*n/a*			
quasi-Newton (LMVM, history 100)	`lmvm`	17	17	17	17
bounded quasi-Newton + line	`bqnls`	30	76	130	163
nonlinear conjugate gradient	`cg`	1180†	3758†	— *(see caption)*	
orthant-wise quasi-Newton	`owlqn`	*n/a*			

Table 9: TAO solver iteration counts on the Poisson optimal control problem of Section 10.7, on the deterministic graded meshes of Table 6, at convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$. `lmvm` uses the Section 10.5 fix ($-tao\_lmvm\_mat\_lmvm\_hist\_size = 100$). `bnk` requires explicit bound vectors and so cannot run on the unbounded problem. `owlqn` is designed for L^1-regularised objectives; it reports immediate convergence at $m = 0$ under its L^1 optimality condition, which is not the optimum of the L^2 Tikhonov problem. †Unlike the other solvers, `cg` does not converge within PETSc's default function-evaluation budget: on every mesh it stops at the `DIVERGED_MAXFCN` limit after about 450 iterations, which is the cap on function evaluations rather than the 2000-iteration cap. With the budget raised it does converge, but the count grows rapidly with refinement, from 1180 at $r = 4$ to 3758 at $r = 16$; at $r = 64$ and $r = 128$ it runs to several thousand iterations and was not taken to completion. This confirms that `cg` is strongly mesh-dependent, in contrast to the flat Newton-class and `lmvm` rows.

The same survey was attempted on the one-dimensional Burgers problem, again on the synthetic exponentially-stretched interval; that setup is invalid for the reason given above, and its results are not included. Rerunning it on a non-collapsing grading scheme, or on a two-dimensional Burgers problem using the same deterministic meshes as the Poisson problem, remains to be done. The expected outcome, by analogy with the Poisson survey of Table 9, is that the Newton-class and `lmvm` solvers are flat across the grading while `cg` is mesh-dependent.

10.10 Discussion

Panel (a) reproduced the central finding of Schwedes et al. [13, Sect. 2.3]: when the function-space inner product is respected, the quasi-Newton iteration count is essentially mesh-independent, whereas under the ambient ℓ^2 inner product it grows at the polynomial rate predicted in Schwedes et al. [13, Sect. 2.2.4]. Three observations on the optimisation packages follow.

First, specifying `riesz_map` on the `Control` is enough for TAO/LMVM. Contrary to a conjecture recorded in an earlier version of this chapter, the LMVM matrix does respect the inner product induced by its initial-Hessian preconditioner in its rank-two updates; the original failure to reproduce the Moola row was caused by the limited-memory approximation being insufficient, not by an internal ℓ^2 inner product. The diagnostic chain is given in Section 10.9: `lmvm` carries the mass-matrix seed in every run, so its growth at the default history of five and its mesh-independence at history 100 $(17, 17, 17, 17)$ isolate the history as the cause of the LMVM failure. The related `bqnls` solver, for which pyadjoint installs no such seed, is mesh-dependent even at a large history until the seed $J_0 = M_h$ is added by hand, a separate confirmation that the metric must be carried by the seed. The original history sweep at $h \in \{5, 25, 50, 100, 200\}$ used the invalid synthetic stretch and is no longer included; repeating it on the deterministic meshes remains to be done. The practical recipe for reproducing the Moola row through stock

TAO/LMVM is therefore to construct `Control(m, riesz_map="L2")` and to set `tao_lmvm_mat_lmvm_hist_size` to a value that captures the relevant part of the discrete Hessian spectrum.

Second, the hand-written Hilbert-space L-BFGS retains its value even though TAO/LMVM is in principle sufficient. The implementation of Section 10.6 serves as a reference in which every dot product of the two-loop recursion is explicit in the Python source, so it is straightforward to verify where the chosen inner product enters. It also avoids the pyadjoint options-prefix subtlety that makes the LMVM history awkward to set through the public `parameters` interface (see Section 10.9).

Third, the apparent discrepancy between our Panel (b) and Schwedes et al. [13, Table 2.2(b)], noted in an earlier version of this chapter, is resolved. The large iteration counts recorded there arose from applying the H^1 metric to the L^2-regularised objective of Panel (a), a mismatch between the metric and the norm in which the problem is posed. Schwedes' Panel (b) regularises in H^1, matching its metric (Section 10.8); when the same match is made here, the Hilbert-space L-BFGS is mesh-independent at four iterations across the sweep, reproducing the constant Moola row exactly.

11 Source-Level Comparison of the L-BFGS Implementations

This section sets the limited-memory BFGS of PETSc TAO beside the two formulations of Schwedes et al. [13, Sect. 1.5.4], the Euclidean and the Hilbert-space, and beside the Moola reference implementation, at the level of the actual recursion. The purpose is to justify the claim of Section 10.10 that TAO/LMVM performs the mathematically correct function-space optimisation, by identifying *where* an inner product enters each algorithm and confirming that the four agree once that single choice is fixed. The source was read against PETSc `v3.24.0` and Moola's reference L-BFGS implementation;[4] the structural conclusions are version-independent, though the specific line references are not.

11.1 The common skeleton

All four algorithms are the same limited-memory two-loop recursion for the action $H_k\, g$ of the inverse-Hessian approximation on the gradient; none forms H_k as a matrix, in the sense that only the action is required (cf. Schwedes et al. [13, Sect. 1.5.4.2]). They differ on exactly one axis, the inner product, which enters in exactly one place: the initial inverse Hessian H_0, the *seed*, applied between the two sweeps. Every other operation in the recursion pairs a gradient object with a step object and carries no metric. Written so that the seed is isolated, the shared recursion is

```
function direction(g):            # returns -H_k g
    q = g
    for j = newest ... oldest:    # first sweep
        a[j] = rho[j] * <s[j], q>     # pairing: step . q
        q    = q - a[j] * y[j]
    z = H0(q)                     # SEED: the one metric step
    for j = oldest ... newest:    # second sweep
        b    = rho[j] * <y[j], z>     # pairing: grad-diff . z
        z    = z + (a[j] - b) * s[j]
    return -z
```

The four algorithms instantiate the pairing `<.,.>`, the curvature scalar `rho` and the seed `H0` differently; that is the entire content of the comparison.

[4] Specifically `src/ksp/ksp/utils/lmvm/` in PETSc and `moola/algorithms/bfgs.py` in Moola.

11.2 The two Schwedes formulations

In the Euclidean, or $\mathbb{R}^n$, formulation [13, (1.187), (1.209)] the control is a coefficient vector, every pairing is the Euclidean dot product, and the seed is a scaled identity. The curvature scalar is $\rho_k = 1/(y_k^\mathsf{T} s_k)$, the sweep terms are $s_j^\mathsf{T} q$ and $y_j^\mathsf{T} z$, and the seed is $H_0 = \gamma_k I$ with $\gamma_k = (s_{k-1}^\mathsf{T} y_{k-1})/(y_{k-1}^\mathsf{T} y_{k-1})$; the denominator $y^\mathsf{T} y$ is a dot of two derivative objects with no inverse between them.

In the Hilbert-space formulation [13, (1.205), (1.210)] the control lives in M, the derivative $\mathrm{d}\tilde{J}(m_k) \in M^*$ is a cofunction, and the pairings are duality pairings. The curvature scalar is $\rho_k = 1/y_k(s_k)$ and the sweep terms are $q(s_j)$ and $y_j(z)$, a cofunction evaluated on a function. The operator V_k of (1.191) splits into the primal and dual operators $\mathcal{V}_k$ and $\mathcal{V}_k^*$ of (1.206)–(1.207), because M and M^* are distinct spaces, and the seed H_0 must be an operator $M^* \to M$, in practice a scaled Riesz map $\gamma_k \mathcal{R}_H$ once an inner product is chosen.

The only structural difference between the two is the seed, and Schwedes et al. [13, (1.208) and following] states this outright: with $M = \mathbb{R}^n$ and $(a,\, b) = a^\mathsf{T} b$ the Riesz map is the identity and the Hilbert update reduces to the Euclidean one, whereas for $L^2(\Omega)$ or $H^1(\Omega)$ controls it does not. The hand-written reference of Section 10.6 writes the Hilbert algorithm in its *primal-representer* form: it pre-applies $\mathcal{R}_H$ to the derivative, so that $g_k := \mathcal{R}_H(\mathrm{d}\tilde{J}(m_k))$ is primal, and takes all pairings in $(\cdot,\, \cdot)_H$. This is identical to the dual, pairing form above, since pre-applying $\mathcal{R}_H$ and using $(\cdot,\, \cdot)_H$ is the same as keeping the derivative dual and using $\mathcal{R}_H$ only as the seed; the two are the two sides of the same coin of the Riesz representation theorem (Section 2.4).

Two typographical slips appear in the printed equations. Equation (1.205) of Schwedes et al. [13] prints the final rank-one term as $+\, m^*(s_k)\, s_k$, without the denominator $y_k(s_k)$; by consistency with the $\rho_k s_k s_k^\mathsf{T}$ term of (1.187), in which $\rho_k = 1/y_k^\mathsf{T} s_k$, the type-correct term requires the factor $1/y_k(s_k)$, and both Moola (Section 11.4) and the limited-memory expansion carry it. Equation (1.210) prints the last denominator with s_{k-2}; by consistency with the last Euclidean term $\rho_{k-1} s_{k-1} s_{k-1}^\mathsf{T}$ of (1.209) it should read $y_{k-1}(s_{k-1})$. Both are recorded as observed on the page, to be raised rather than silently corrected.

11.3 What TAO/LMVM computes

TAO's LMVM driver (`tao/.../lmvm/lmvm.c`, `TaoSolve_LMVM`) is a standard quasi-Newton loop: compute the gradient, build a direction from memory, check that it is a descent direction, perform a line search, and repeat. The memory (`symbrdn/symbrdn.c`, `MatUpdate_LMVMSymBrdn`) forms $s = X - X_{\mathrm{prev}}$, a primal step, and $y = F - F_{\mathrm{prev}}$, the assembled (hence dual) gradient difference, and stores the pair only if $s^\mathsf{T} y > 0$. The direction (`bfgs/bfgs.c` and `dfp/dfp.c`) is the two-loop recursion, with two implementation quirks. The BFGS solve $H_k g$ is computed by the DFP kernel in dual mode, exploiting the BFGS/DFP duality, so the literal s and y roles are reordered relative to a standard two-loop although the operator is the same; and the sweep loops are batched, a loop of dot products becoming a single `GEMVH` call over the stored basis and a loop of updates a single `GEMV`. Every `VecDot` or `GEMVH` pairs the dual y, or the gradient, with a primal s, or the running vector; these are duality pairings, equal to the ℓ^2 coefficient dot, so the bare `VecDot` is the *correct* pairing and carries no metric. The seed H_0 is applied by `MatLMVMApplyJ0Inv`, a `KSPSolve` on J_0: with $J_0 = M_h$, the mass matrix installed by `Control(m, riesz_map="L2")`, this solves $M_h z = q$, so that $z = M_h^{-1} q = \mathcal{R}_{L^2}(q)$ and the seed *is* the L^2 Riesz map, whereas with no J_0 it is the scalar $\gamma_k I$. When J_0 is a non-diagonal matrix, TAO's scale-type logic sets the symmetric-Broyden scaling to `NONE`, so the Euclidean $\gamma_k = s^\mathsf{T} y / y^\mathsf{T} y$ is not applied and the Riesz seed M_h^{-1} is used unscaled. The placement of the metric is therefore clear: the default is Euclidean (Schwedes et al. [13, (1.209)], mesh-dependent), while $J_0 = M_h$ gives the Hilbert update (Schwedes et al. [13, (1.210)], mesh-independent).

11.4 What Moola computes

Moola keeps `DualVector` and `PrimalVector` as distinct types: `x.apply(s)` is the duality pairing and `x.primal()` is the Riesz map. Its recursion (`matvec`) is the recursive form of Schwedes et al. [13, (1.210)], with pairings taken through `.apply()` and the rank-one term carrying $\rho = 1/y(s)$ explicitly, which independently confirms the (1.205) denominator discussed above. The seed is `Hinit = dual_to_primal = .primal()`, the Riesz map *by default*, the exact opposite of TAO's default. The seed scaling `theta` $= y(s)/y\big(y.\texttt{primal()}\big) = (s,\, y)/(y,\, y)_{M^*}$ uses the *dual* norm, with the inverse inside, unlike the Euclidean γ_k. Convergence is tested in the dual norm `primal_norm()` $= \|g\|_{M^*}$ by default.

11.5 Side-by-side instantiations

The same skeleton with each algorithm's specific choices substituted:

```
# --- Schwedes Euclidean (Eq 1.187 / 1.209) ---
function direction_schwedes_Rn(g):
    q = g                                   # g, s, y all in R^n
    for j: a[j] = (s[j].T @ q) / (y[j].T @ s[j])   # Euclidean dot
           q   -= a[j] * y[j]
    gamma_k = (s[k-1].T @ y[k-1]) / (y[k-1].T @ y[k-1])
    z = gamma_k * q                         # SEED: gamma_k * I
    for j: b   = (y[j].T @ z) / (y[j].T @ s[j])
           z  += (a[j] - b) * s[j]
    return -z
```

```
# --- Schwedes Hilbert (Eq 1.205 / 1.210) ---
function direction_schwedes_H(g):
    q = g                                   # g, y in M^*; s in M
    for j: a[j] = q(s[j]) / y[j](s[j])      # duality pairing
           q   -= a[j] * y[j]
    z = gamma_k * R_H(q)                    # SEED: scaled Riesz
    for j: b   = y[j](z) / y[j](s[j])
           z  += (a[j] - b) * s[j]
    return -z
```

```
# --- Moola (recursive form of Eq 1.210) ---
function direction_moola(g):
    q = g                                   # dual
    for j: a[j] = s[j].apply(q) / y[j].apply(s[j])
           q.axpy(-a[j], y[j])
    theta = y[k-1].apply(s[k-1]) / y[k-1].apply(y[k-1].primal())
    z = theta * q.primal()                  # SEED: theta * Riesz
    for j: b   = y[j].apply(z) / y[j].apply(s[j])
           z.axpy(a[j] - b, s[j])
    return -z
```

```
# --- TAO LMVM (with Control(m, riesz_map="L2")) ---
function direction_tao_lmvm(g):
    q = g                                   # PETSc Vec of coefficients
    for j: a[j] = VecDot(s[j], q) * rho[j] # ell^2 = duality pairing
           VecAXPY(q, -a[j], y[j])
    z = M_h^{-1} q                          # SEED: KSPSolve(J_0=M_h)
    for j: b   = VecDot(y[j], z) * rho[j]
           VecAXPY(z, a[j] - b, s[j])
    return -z
```

Reading down the column of seed lines, only those lines differ; the sweeps are structurally identical. The Schwedes-Euclidean and Schwedes-Hilbert variants differ only in whether the

seed is $\gamma_k I$ or $\gamma_k \mathcal{R}_H$, consistent with Schwedes et al. [13, (1.208)] reducing one to the other when $\mathcal{R}$ is the identity. Moola and TAO-with-L^2 differ only in the scalar θ against an unscaled seed: Moola scales the Riesz seed by a dual-norm-correct θ, while TAO uses the bare Riesz seed because its scalar rescaling is auto-disabled when J_0 is a matrix (Section 11.3).

11.6 Side-by-side comparison

Item	Schwedes $\mathbb{R}^n$	Schwedes Hilbert	Moola	TAO (`v3.24`)
step s_k	$x_{k+1} - x_k$	$m_{k+1} - m_k \in M$	αp_k (primal)	$X - X_{\text{prev}}$
grad diff y_k	$\nabla f_{k+1} - \nabla f_k$	$\mathrm{d}\tilde{J}_{k+1} - \mathrm{d}\tilde{J}_k \in M^*$	$g_{k+1} - g_k$ (dual)	$F - F_{\text{prev}}$
curvature	$y_k^{\mathsf{T}} s_k$	$y_k(s_k)$ pairing	`y.apply(s)`	`VecMDot`
sweep pairings	Euclidean dot	duality pairing	`.apply()`	`VecDot`
recursion	two-loop	two-loop $(\mathcal{V}_k, \mathcal{V}_k^*)$	recursive	batched, DFP-dual
seed (default)	$\gamma_k I$	$\gamma_k \mathcal{R}_H$	$\mathcal{R}$ (Riesz)	$\gamma_k I$
seed ($\mathcal{R}$ set)	—	$\mathcal{R}_H$	$\mathcal{R}$	M_h^{-1}
seed scaling	via $y^{\mathsf{T}} y$	via $(y,\, y)_{M^*}$	dual-norm	off if $J_0{=}M_h$
conv. norm	ℓ^2	dual	dual	ℓ^2 default

Table 10: The four limited-memory BFGS implementations differ only in how the single inner-product choice is realised. Every row above the seed is a metric-free pairing; the algorithms diverge at the seed H_0 and in the (convergence-only) norm.

11.7 Is anything wrong with TAO/LMVM?

Structurally, no. The recursion contains no cofunction-with-cofunction dot product missing an inverse; every pairing is dual-with-primal, and the metric enters only at the seed. With $J_0 = M_h$ the seed is the L^2 Riesz map, and TAO realises Schwedes et al. [13, (1.210)] exactly, the same operator as Moola and as the reference of Section 10.6. The empirical isolation of Table 9 supports this: `lmvm`, which carries the mass-matrix seed, is mesh-dependent at the default history and mesh-independent with the history-size fix $(17, 17, 17, 17)$, so the limited-memory truncation, rather than any inner-product bug, is the cause of the original LMVM failure. The related `bqnls` solver reinforces the role of the seed itself: pyadjoint installs none for it, and it stays mesh-dependent until the seed $J_0 = M_h$ is added by hand (Section 10.9), which directly shows the point of this section, that the metric enters at H_0.

The correctness is nonetheless conditional on premises the recursion assumes rather than guarantees. The key one is that the gradient reaches TAO as a dual object and s as a primal one: the argument that `VecDot` is the right pairing fails if pyadjoint pre-applies the Riesz map and hands TAO a primal gradient, as the reference of Section 10.6 deliberately does, for then the same dots become primal-with-primal and *are* missing an inverse. This is a fact about the pyadjoint interface rather than the TAO source, and is the one premise that could invert the verdict, so it is worth confirming directly. The remaining premises are milder. The Riesz solve must be exact: M_h^{-1} is applied by a `KSPSolve` on J_0, which a direct LU makes exact, whereas an iterative solve with a weak preconditioner applies a different operator and the metric is no longer clean (the same caution as for the Riesz solve in Section 10.6). The space must be right: $J_0 = M_h$ gives the L^2 Riesz map only, while an H^1 control needs $J_0 = M_h + K_h$, a Helmholtz solve. And the variant must be right: the analysis is for `MATLMVMBFGS` in the `recursive` multiplication mode, whereas the `compact_dense` and `dense` variants form Gram-type products $Y^{\mathsf{T}} Y$ that reintroduce cofunction-cofunction dots and would need separate checking.

Even when the algorithms are matched, residual differences remain, though these affect the

iteration count rather than correctness. Moola scales its Riesz seed by a dual-norm-correct `theta` while TAO uses M_h^{-1} unscaled; Moola tests convergence in the dual norm by default while TAO uses ℓ^2 unless `TaoSetGradientNorm` is set, which changes *when* the solver stops rather than its iterates; and, although both use a Moré–Thuente strong-Wolfe line search, so this is not itself a source of difference, their default history lengths differ, five against ten. The practical failure was therefore not a correctness bug: the original inability to reproduce the Moola row (Section 10.9) was the limited-memory history being too short for the discrete-Hessian spectrum on graded meshes, not an internal ℓ^2 inner product.

11.8 Notes useful for the project

Several practical points follow. To make TAO reproduce Moola, construct `Control(m, riesz_map="L2")`, which installs $J_0 = M_h$, and enlarge `tao_lmvm_mat_lmvm_hist_size`; for an exact match, also match the history length, the line search, and, if iteration counts are to be compared, the convergence norm and the seed scaling. It is `setLMVMH0` that matters, not `setGradientNorm`: the seed, and hence J_0, determines the search direction and the mesh-(in)dependence, while `TaoSetGradientNorm` only weights the convergence test. This resolves the apparent conflict between the two mechanisms noted in the meeting record. The recursion is matrix-free except at the seed: it never assembles H_k, and the one genuine linear solve is the seed M_h^{-1}, which is also why, unlike Newton/NLS, there is no inner Krylov solve to precondition: the metric must be injected at H_0 rather than as a preconditioner. As a version note, the `v3.24` LMVM code is the refactored `LMBasis`/`LMProducts` form, with batched `GEMV` and shared DFP/BFGS kernels; the mathematics is unchanged from the explicit two-loop of earlier releases, but any line reference should cite the file matching the installed PETSc. Finally, one empirical leg remains open: the source argument predicts mesh-independence, but demonstrating it fully requires the *randomly*-refined meshes of Schwedes et al. [13, Sect. 2.3.2], with ratios up to about 10^5, rather than only the graded meshes used here, and this is the natural companion to the future-work item of Section 13.

12 Two-Level Riesz-Map Preconditioning

The experiments of Section 10 reproduced the finding of Schwedes et al. [13] that a quasi-Newton optimiser becomes mesh-independent once its inner product (equivalently, the initial-Hessian seed of its limited-memory recursion) is the Riesz map of the control space. That analysis concerns the outer optimiser alone. This section develops the following observation. The same mesh-dependence appears again one level deeper, in the inner Krylov solve that a Newton optimiser uses to compute each search direction, and it is removed by the same idea applied there, by preconditioning that inner solve with the Riesz map. That the inner product plays the role of a preconditioner is anticipated by classical operator preconditioning [6, 8] and was raised in a discussion among the PETSc/TAO developers,[5] but we expose and cure it experimentally, which to our knowledge has not been done before in an adjoint optimisation pipeline built on pyadjoint. The metric must therefore be injected at two levels, as the quasi-Newton seed on the outer iteration and as the preconditioner on the inner conjugate-gradient solve, and mesh-independence requires it at both. We first expose the hidden inner-level growth and remove it with an L^2 Riesz-map preconditioner, of which a productionised version has been prepared for the pyadjoint library, and then show that the preconditioner must be built from the Riesz map that matches the regularisation, verifying that the residual mesh-dependence of a mismatched map is genuine conditioning.

[5] PETSc/TAO developer discussion on inner products, Riesz maps and mesh-independent convergence, on the Firedrake/PETSc Discord, March 2025 [7]; in particular the observations that, for linear solvers, "changing the inner product" is exactly preconditioning, and that an inexact inner solve needs a mesh-independent preconditioner together with a preconditioned norm.

12.1 What preconditioning does

Before turning to the experiments, we recall what preconditioning is. An iterative linear solver such as conjugate gradients does not invert the system matrix A in $Ax = b$; it takes a sequence of cheap steps that progressively drive the residual to zero. The number of steps it needs is governed by the *conditioning* of A: loosely, by how widely its eigenvalues are spread. When the eigenvalues are tightly clustered the solver converges in a handful of steps; when they span many orders of magnitude it takes many. Preconditioning replaces the system by an equivalent one, $P^{-1}Ax = P^{-1}b$, with the same solution but a better-conditioned operator, so the solver converges far faster. The preconditioner P must be cheap to apply and should approximate A well enough that $P^{-1}A$ has clustered eigenvalues; taking $P = A$ would converge in a single step but is as costly as the original problem, so the trick is to capture the source of the poor conditioning while staying cheap. In our case that source is known exactly: it is the cell-size scaling that the mass matrix introduces on a non-uniform mesh, and the operator that cancels it is the Riesz map.

12.2 Preconditioning the inner CG of TAO/NLS

The `nls` row of Table 9 reports two outer iterations on every mesh in the sweep and so appears mesh-independent. This appearance hides a mesh-dependence one level deeper. At each outer Newton step, `nls` solves $Hp_k = -g_k$ for the search direction by an inner Krylov iteration, conjugate gradients by default, where H is the discrete reduced Hessian and g_k the assembled dual gradient. Expressed in the ambient ℓ^2 coefficient inner product, H is badly conditioned on graded meshes for the same reason the outer L-BFGS Hessian is (Section 10.7): the cell-size scaling encoded in the diagonal of the mass matrix. The inner CG iteration count per outer Newton step therefore grows with the grading, even though the outer Newton count does not. This is Schwedes' pathology repeated one level down.

To measure the hidden growth, we ran `nls` on the Poisson optimal control problem of Section 10.7, on the six deterministic graded meshes of Table 6, with no preconditioner on the inner KSP (`-tao_nls_ksp_pc_type none`), and installed a counting monitor on TAO's inner KSP to sum the CG iterations across both outer Newton steps. The totals, reported in the baseline column of Table 11, grew from 61 at $r = 4$ to 1052 at $r = 128$, roughly doubling at each refinement.

The same reasoning as in Section 10.7 applies: preconditioning the inner CG with the discrete Riesz map $\mathcal{R}_{L^2} = M_h^{-1}$ cancels the cell-size scaling that ruined the conditioning. The module `meshdep.preconditioners` provides an `AuxiliaryOperatorPC` subclass `RieszMapL2` that returns the mass form, and the natural recipe is a single options-database entry:

```
'tao_nls_ksp_pc_type': 'python',
'tao_nls_ksp_pc_python_type':
    'meshdep.preconditioners.RieszMapL2',
```

This silently failed, however. The `AuxiliaryOperatorPC` needs the operator matrix to carry a Firedrake function space so that it can build the mass form on the correct space, whereas pyadjoint's reduced-Hessian matrix is a Python `Mat` without one; the preconditioner was left at type `none` with no warning. The workaround, `meshdep.preconditioners.RieszMapPCContext`, is a plain PETSc Python preconditioner that takes the function space at construction, assembles M_h from it and factorises it by LU. It is wired directly onto the inner KSP after the `TAOSolver` is built:

```
solver = TAOSolver(MinimizationProblem(Jhat), parameters=...)
pc = solver.tao.getKSP().getPC()
pc.setType('python')
pc.setPythonContext(RieszMapPCContext(V, riesz_map="L2"))
```

```
solver.solve()
```

The diagnostic `tao_nls_ksp_view` then reported `PC type: python` with the Riesz context attached, confirming that the preconditioner was installed.

The baseline and Riesz-L^2 runs on the same six meshes are compared in Table 11 and plotted in Figure 11. Outer Newton stayed at two iterations in every row, since the Poisson problem is quadratic, so the entire row-to-row variation lies in the inner CG. The baseline totals grew $61 \to 179 \to 573 \to 1052$ across $r \in \{4, 16, 64, 128\}$, matching the qualitative shape of the SciPy ℓ^2 row of Table 7, while the Riesz-L^2 totals were flat at 12 CG iterations on every mesh in the sweep, a reduction of roughly $88\times$ at $r = 128$.

target r	realised $h_{\max}/h_{\min}$	cells	baseline inner CG	Riesz L^2 inner CG
4	4.00	256	61	12
16	16.00	1 712	179	12
64	64.00	22 608	573	12
128	128.00	87 312	1052	12

Table 11: Total inner-CG iteration counts across both outer Newton steps for TAO's `nls` on the Poisson optimal control problem, on the deterministic graded meshes of Table 6, at convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$. Outer Newton took two iterations in every row. The baseline column uses no preconditioner on the inner KSP; the Riesz-L^2 column uses a plain PETSc Python preconditioner applying M_h^{-1} by a direct LU factorisation of the mass matrix on the control space.

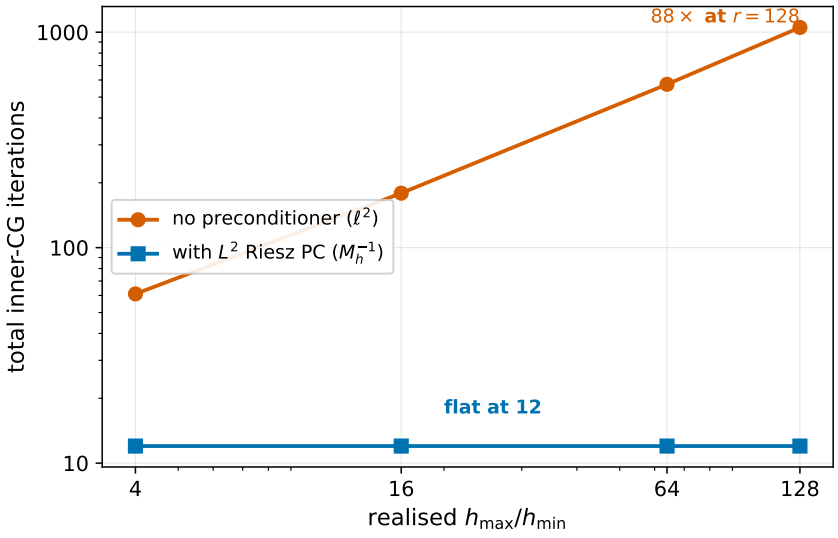

Figure 11: Total inner-CG iterations for TAO's `nls` on the Poisson optimal control problem against the realised mesh ratio, on the deterministic graded meshes of Table 6 (the data of Table 11). Without a preconditioner the count grows polynomially with the mesh ratio; with the L^2 Riesz-map preconditioner it stays flat at twelve.

Table 11 is the direct analogue of Table 7 at the level of the inner Krylov solve rather than the outer optimiser: the same pathology, the bad ℓ^2 conditioning of a discrete operator on graded meshes, is cured by the same Riesz map, applied here as a Krylov preconditioner rather than as a quasi-Newton initial-Hessian seed. It also closes the loose end in the `nls` row of Table 9,

whose flat outer counts were only half of the story. Repeating the experiment with the H^1 Riesz map `RieszMapH1`, also available in `meshdep.preconditioners`, on the H^1-regularised problem of Section 10.8, is a natural extension that remains to be carried out.

12.3 A productionised preconditioner for pyadjoint

The `RieszMapPCContext` used above takes the control function space explicitly at construction, which is fine for a single experiment but awkward as a reusable component. The main software contribution of this work is a general version, `RieszMapPC`, that reads the Riesz map directly from each `Control` and so needs no such argument. Because it applies whichever Riesz map a control carries, the one preconditioner serves both the L^2 and the H^1 problem without modification. It is written against the TAO solver interface now being added to pyadjoint, which is what will let Firedrake drive optimisation through PETSc TAO, and is implemented as a subclass of petsctools' `PCBase`, guarded by a fallback so that it stays importable in environments without `petsc4py` and raises only when actually used. It is selected through the two options `tao_nls_pc_type = python` and `tao_nls_pc_python_type = pyadjoint.optimization.tao_solver.RieszMapPC`.

The preconditioner is covered by a test that installs it on TAO's inner Krylov solve and checks three things: that it is actually attached, with the diagnostic reporting `PC type: python` and the Riesz context present; that it drives the inner-CG count below the unpreconditioned baseline; and that it reproduces the flat count of the hand-wired `RieszMapPCContext` exactly, on the same problem. On the Poisson problem of this section it matches Table 11 to the iteration. The code sits on a branch of pyadjoint prepared for a pull request upstream, so that the two-level fix becomes available to any pyadjoint user through the metric already attached to their `Control`, rather than through the manual plumbing of Section 12.2.

12.4 Why the Riesz map removes the mesh-dependence

The flat count in Table 11 is not particular to this problem; it follows from the spectrum of the operator that the inner CG solves. In the L^2 inner product the reduced Hessian is $\mathcal{H} = S^*S + \alpha I$, where S is the control-to-state solution map. Because S solves a Poisson problem it is smoothing, so S^*S is compact and its eigenvalues decay to zero. The operator $\mathcal{H}$ is therefore the identity scaled by α plus a compact perturbation, and its eigenvalues cluster at α with only finitely many outliers. Its conditioning is bounded independently of the mesh, and that is exactly the regime in which conjugate gradients converges in a fixed number of steps.

What the inner CG sees without a preconditioner, though, is not $\mathcal{H}$ but its matrix in the ℓ^2 coefficient inner product, which carries an extra factor of the mass matrix M_h. On a graded mesh the diagonal of M_h ranges over the cell areas, so M_h is badly conditioned, and this scaling contaminates the spectrum and makes the CG count grow. Preconditioning by M_h^{-1}, the discrete L^2 Riesz map, removes exactly this factor and returns the CG to the mesh-independent spectrum of $\mathcal{H}$ itself. This is the finite-element instance of a general principle: the natural preconditioner for a discretised operator is the discrete Riesz map of the space on which it acts [6, 8].

The same picture explains why the H^1 map fails on this problem. Preconditioning the L^2-natural operator by the H^1 Riesz map $(M_h + K_h)^{-1}$ does not cancel the mass matrix; it over-smooths, replacing the clustered spectrum of $\mathcal{H}$ by one whose smallest eigenvalues shrink as the mesh resolves higher frequencies, so the count grows again. Mesh-independence returns only when the regularisation is itself an H^1 norm, as in Section 12.5, because the H^1 Riesz map is then the matching one.

12.5 Matching the Riesz map to the regularisation

The inner-CG preconditioning of Section 12.2 used the L^2 Riesz map, which is the correct choice for the L^2-regularised problem. To see that the choice must *match* the regularisation, we ran the same TAO/NLS solve on the interior random-refined meshes of Section 10.4, preconditioning the inner CG in turn with the L^2 map M_h^{-1} and the H^1 map $(M_h + K_h)^{-1}$. The L^2-regularised results are given in Table 12, and Figure 12 plots them beside the mirror experiment described below.

level	cells	baseline	L^2 PC	H^1 PC
4	314	65	14	131
6	772	108	13	205
8	1 970	341	13	323
10	5 450	614	14	417
12	16 020	1105	13	522
14	46 998	2115	14	526

Table 12: Total inner-CG iteration counts for TAO's `nls` on the L^2-regularised Poisson optimal control problem, over six interior random-refined meshes spanning realised $h_{\max}/h_{\min}$ comparable to the graded meshes of Table 6, with no preconditioner (baseline) and with the L^2 and H^1 Riesz maps as inner-KSP preconditioners. Outer Newton took two iterations in every case. The L^2 map is flat; the H^1 map, mismatched to the L^2 regularisation, grows almost as fast as the baseline.

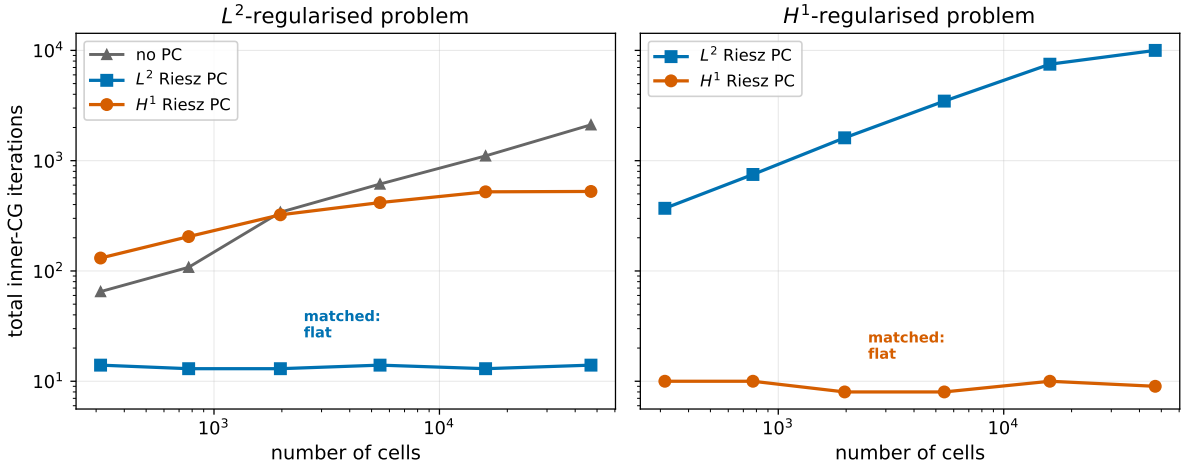

Figure 12: Total inner-CG iterations for TAO's `nls` against the number of mesh cells, on the interior random-refined meshes, under the L^2 and H^1 Riesz-map preconditioners. *Left:* the L^2-regularised problem (the data of Table 12). *Right:* the H^1-regularised problem. In each case the map that matches the regularisation stays flat and the mismatched map grows, so the same two colours swap roles when the norm changes.

The L^2 map keeps the inner-CG count flat, reproducing on random meshes the behaviour of Table 11, while the H^1 map, mismatched to the L^2 regularisation, grows with the mesh, from 131 to 526 over the range, almost as fast as the unpreconditioned baseline.

Two checks confirm that this growth is genuine conditioning rather than an artefact of early termination. First, the L^2- and H^1-preconditioned solves reach the *same* minimiser: at the third

refinement level the two objective values agree to fifteen digits ($|J^*_{L^2} - J^*_{H^1}| = 3 \times 10^{-15}$) and the controls differ in relative L^2 norm by under 10^{-6}, while the inner-CG counts differ by a factor of twenty-five (13 against 323). Second, interpolating between the two maps by preconditioning with $M_h + wK_h$ and sweeping the stiffness weight w shows that the mesh-dependence is present for *every* positive weight and vanishes only at $w = 0$: at the finest mesh the count falls from 672 at $w = 1$ to 73 at $w = 10^{-4}$, but each fixed positive w still grows with the mesh, and only the pure L^2 map is flat. The H^1 term is therefore genuinely the wrong norm for this problem, not merely a poorly scaled one.

The converse holds when the regularisation is changed to match. On an H^1-regularised problem (the objective of Section 10.8), the roles reverse: the H^1 Riesz map keeps the inner-CG count flat, between 8 and 10 across the sweep, while the L^2 map now grows, from 370 to the 10^4-iteration cap. This is the inner-CG counterpart of the outer-optimiser result of Section 10.8, and it makes the same point from the preconditioning side: the Riesz map that delivers mesh-independence is the one matching the norm in which the control is regularised.

12.6 Using the preconditioner

The two-level preconditioning developed above needs none of the manual plumbing of Section 12.2 to reuse. On the outer optimiser the metric is selected by the `riesz_map` argument of the pyadjoint `Control`; on the inner Krylov solve the productionised `RieszMapPC` is selected through two PETSc options. A complete inner-preconditioned Newton solve of a control problem reads:

```
from firedrake.adjoint import Control, ReducedFunctional
from pyadjoint import MinimizationProblem
from pyadjoint.optimization.tao_solver import TAOSolver

c = Control(m, riesz_map="L2")          # metric on the outer optimiser
Jhat = ReducedFunctional(J, c)

solver = TAOSolver(MinimizationProblem(Jhat), {
    "tao_type": "nls",
    "tao_nls_pc_type": "python",
    "tao_nls_pc_python_type":
        "pyadjoint.optimization.tao_solver.RieszMapPC",
})
m_opt = solver.solve()
```

The single argument `riesz_map="L2"` sets the outer metric, and the two `tao_nls_pc` options attach the same Riesz map as the inner-CG preconditioner. Changing both to `"H1"` switches the whole solve to the H^1 metric with no other change, which is how the matched and mismatched runs of Section 12.5 were produced.

The experiments of Sections 10 and 12 are driven by a small Python package written for this dissertation, providing the graded and random meshes, the Poisson and Burgers control problems, the four optimiser wrappers used for the panels (SciPy L-BFGS-B, the Hilbert-space L-BFGS, and the two TAO routes), and the research-prototype preconditioners. Each table and figure is produced by a short driver script over these components, and the full package and scripts are available in the accompanying code repository. The experiments were run with Firedrake and its pyadjoint algorithmic-differentiation extension, on top of PETSc, which provides the TAO optimisation toolkit and the Krylov solvers used throughout; the forward and Riesz-map solves use PETSc's direct LU factorisation.

13 Conclusions and Future Work

This work has developed the mathematical and numerical infrastructure for adjoint-based optimal control of two model problems. We derived the weak formulation, the adjoint equation and the L^2-gradient of the reduced functional for the linear Poisson problem in Sections 3 to 5, and verified the resulting numerical pipeline through manufactured solutions and Taylor tests in Section 6. We then extended the same machinery to the nonlinear, time-dependent scalar viscous Burgers problem in Sections 7 to 9, showing that the structure of the adjoint analysis carries over to a problem whose adjoint must be solved backward in time.

Section 10 reproduced Panel (a) of Table 2.2 of Schwedes et al. [13] on six deterministic graded meshes spanning ratios from 4 to 128. A custom implementation of BFGS-in-Hilbert-space following Schwedes et al. [13, Sect. 1.5.4.1], in which every dot product in the two-loop recursion is taken in the L^2 inner product, produced an iteration count that was constant at six across the entire range of mesh ratios. The SciPy L-BFGS-B row, run on the ambient ℓ^2 inner product with the convergence criterion checked externally in L^2, produced an iteration count that doubled at every refinement, in agreement with the polynomial bound derived in Schwedes et al. [13, Sect. 2.2.4]. Routing the same problem through PETSc TAO/LMVM with `Control(m, riesz_map="L2")` also reproduces the qualitative pattern, provided that the LMVM history size is enlarged from the PETSc default of five (which is too small to capture the spectrum of the discrete Hessian on the strongly graded meshes); the diagnostic chain that established this, including a noise-floor probe at machine precision and a Burgers control problem that does not exhibit the same mesh-dependence, is documented in Section 10.9. Panel (b) of Table 2.2, which places the H^1 inner product on the control space, was reproduced once we recognised that its objective regularises in the H^1 norm to match the metric: on the correspondingly H^1-regularised problem the Hilbert-space L-BFGS is mesh-independent at four iterations across the sweep, matching Schwedes et al. [13]'s four, and the large counts seen earlier came from applying the H^1 metric to the L^2-regularised problem rather than from any deficiency of the H^1 Riesz map.

The same mesh-dependence reappears one level deeper, in the inner conjugate-gradient solve of TAO's Newton method, and is removed in the same way, by preconditioning that solve with the L^2 Riesz map: the total inner-CG count falls from 1052 to a flat 12 at the highest mesh ratio. A productionised version of this preconditioner, which reads its Riesz map from each `Control`, has been prepared for contribution to pyadjoint as `RieszMapPC`.

Several directions remain open. The vector-valued time-dependent Burgers problem treated by Schwedes et al. [13] would be a natural extension of Section 8, and would let us reproduce the mesh-dependence experiments of their Chapter 3 on randomly refined meshes. A second direction is to use the inner-product-aware optimisation framework on a problem with practical relevance, for example tidal-turbine array layout optimisation, as in Schwedes et al. [13, Chapter 3]. A third direction, on the implementation side, is to extend the `meshdep` package with Newton-CG and IPOPT wrappers so that the full grid of methods considered by Schwedes et al. [13] is reproducible.

References

[1] Susan C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*, volume 15 of *Texts in Applied Mathematics*. Springer New York, 3 edition, 2008. ISBN 978-0-387-75933-3. doi: 10.1007/978-0-387-75934-0.

[2] Richard H. Byrd, Peihuang Lu, Jorge Nocedal, and Ciyou Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208, 1995. doi: 10.1137/0916069.

[3] Simon W. Funke. moola: an optimisation package for PDE-constrained optimisation. Software, https://github.com/funsim/moola, 2013. Version 0.1.7, GNU LGPL v3. The BFGS default line search is the Moré–Thuente strong-Wolfe `dcsrch` routine from MINPACK-2.

[4] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009. doi: 10.1002/nme.2579.

[5] David A. Ham and Colin J. Cotter. *Finite Elements: Analysis and Implementation*. 2026.0 edition, 2026. Course notes.

[6] Robert C. Kirby. From functional analysis to iterative methods. *SIAM Review*, 52(2): 269–293, 2010. doi: 10.1137/070706914.

[7] Matthew G. Knepley, Tobin Isaac, Jed Brown, Barry F. Smith, Stefano Zampini, Hansol Suh, et al. Inner products, Riesz maps and mesh-independent convergence in PETSc/TAO. Developer discussion, Firedrake/PETSc Discord, March 2025. On Riesz-map (inner-product) awareness for TAO optimisers and preconditioning of the inner Krylov solve.

[8] Kent-Andre Mardal and Ragnar Winther. Preconditioning discretizations of systems of partial differential equations. *Numerical Linear Algebra with Applications*, 18(1):1–40, 2011. doi: 10.1002/nla.716.

[9] Sebastian K. Mitusch, Simon W. Funke, and Jørgen S. Dokken. dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *Journal of Open Source Software*, 4(38): 1292, 2019. doi: 10.21105/joss.01292.

[10] Jorge J. Moré and David J. Thuente. Line search algorithms with guaranteed sufficient decrease. *ACM Transactions on Mathematical Software*, 20(3):286–307, 1994. doi: 10.1145/192115.192132.

[11] Florian Rathgeber, David A. Ham, Lawrence Mitchell, Michael Lange, Fabio Luporini, Andrew T. T. McRae, Gheorghe-Teodor Bercea, Graham R. Markall, and Paul H. J. Kelly. Firedrake: Automating the finite element method by composing abstractions. *ACM Transactions on Mathematical Software*, 43(3):24:1–24:27, 2016. doi: 10.1145/2998441.

[12] María-Cecilia Rivara. Algorithms for refining triangular grids suitable for adaptive and multigrid techniques. *International Journal for Numerical Methods in Engineering*, 20(4): 745–756, 1984. doi: 10.1002/nme.1620200412.

[13] Tobias Schwedes, David A. Ham, Simon W. Funke, and Matthew D. Piggott. *Mesh Dependence in PDE-Constrained Optimisation: An Application in Tidal Turbine Array Layouts*. SpringerBriefs in Mathematics of Planet Earth. Springer International Publishing, 2017. ISBN 978-3-319-59482-8. doi: 10.1007/978-3-319-59483-5.

[14] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.